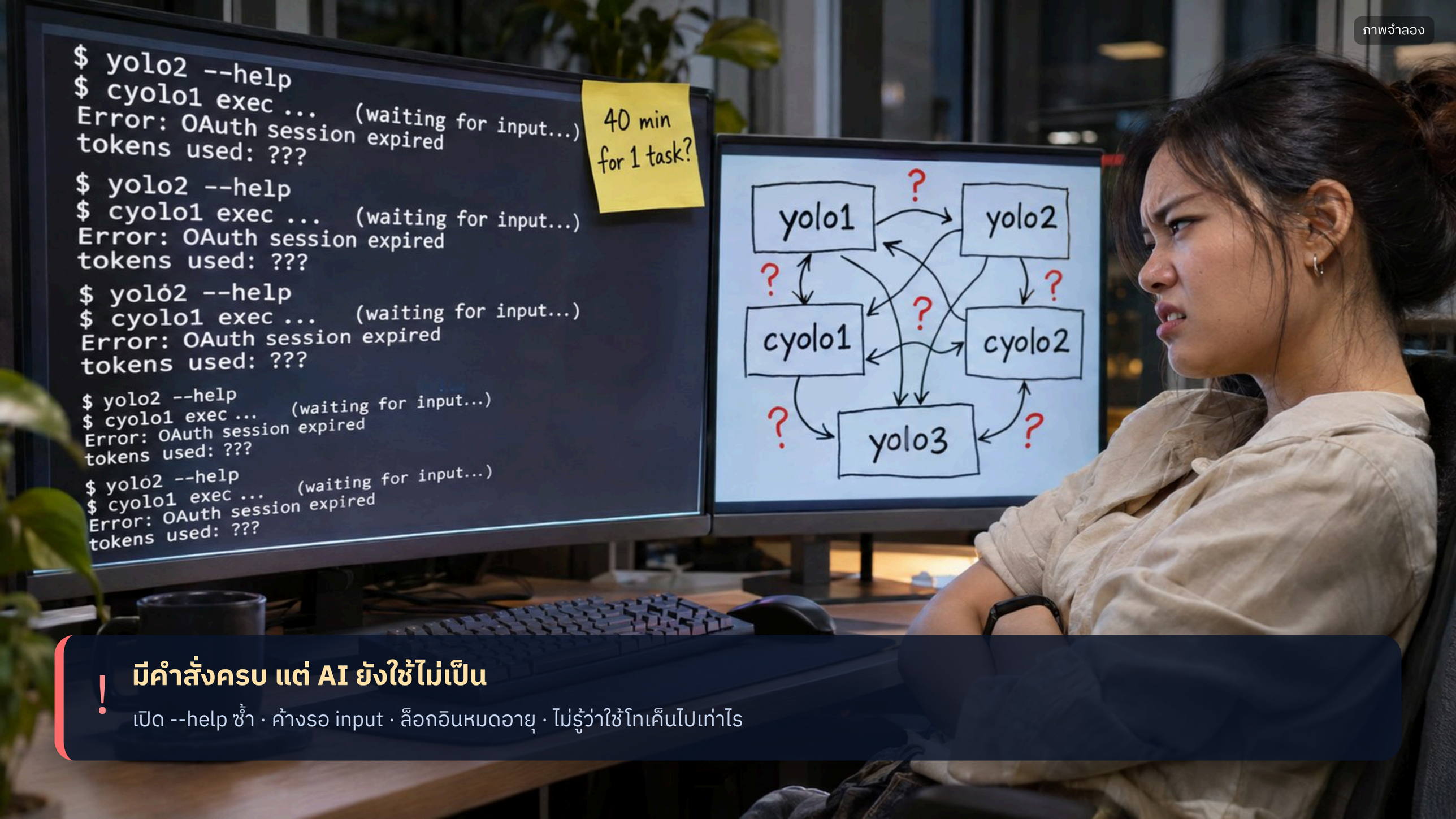
สัปดาห์ที่ 2 · 04_MultiAccount_Skill

การสร้างสกิล สำหรับระบบหลายบัญชี

yolo · cyolo · agent-team

ให้ AI ใช้ **yolo1–5** และ **cyolo1–5** ได้ถูกวิธี · ทำงานเป็นทีม · รู้สถิติทุกบัญชี · สลับหัวหน้าได้เมื่อโควตาหมด





มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น

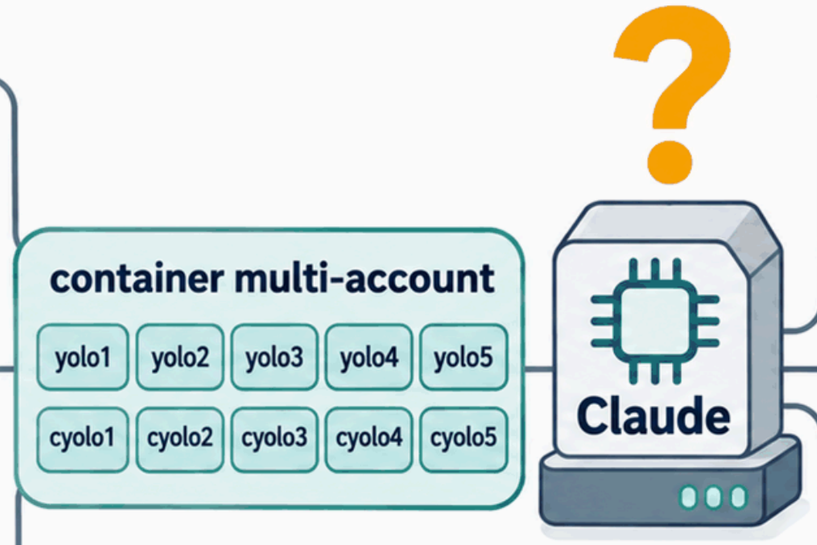
เปิด --help ซ้ำ · ค้างรอ input · ล็อกอินหมดอายุ · ไม่รู้ว่าใช้โทเค็นไปเท่าไร

มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น

เรียกผิดวิธี
ค้างรอ input · เปิดโหมดโต้ตอบ

สถานะล็อกอินไม่ตรงจริง
loggedIn = true แต่หมดอายุ

ประสานงานไม่เป็น
รอทีละตัว · ส่งข้อมูลยาวใน prompt



ไม่มีสถิติ
tokens · เวลา · โควตา ต้องเดา

ลองผิดลองถูกทุกครั้ง
ช้า · เปลืองโทเค็น

หัวหน้าทีมโควตาหมด
แผนงานหาย · งานค้างกลางทาง

1 · ปัญหา

สิ่งที่เกิดขึ้นจริง

เพนพอยต์	สิ่งที่เกิดขึ้นจริง
เรียกผิดวิธี	`codex exec` ไม่ปิด stdin จะค้าง · ไม่ใส่ `-p` เปิดโหมดโต้ตอบ · exit code 0 ≠ สำเร็จ
สถานะล็อกอินไม่ตรงจริง	`auth status` ตอบ loggedIn แต่เรียกจริงได้ `OAuth session expired`
ประสานงานไม่เป็น	สั่งทีละบัญชี · ยัดข้อมูลยาวลง prompt · ไม่มีที่แชร์ความรู้
ไม่มีสถิติ	ไม่รู้โทเค็น เวลา และโควตาของแต่ละบัญชี
ลองผิดลองถูกทุกครั้ง	เปิด --help ซ้ำ เขียนสคริปต์ช่วยใหม่ทุกครั้ง
หัวหน้าทีมโควตาหมด	แผนงานอยู่ในบริบทแชตของหัวหน้า งานค้างหายไปด้วย

1.1 · ทดลองก่อนมีสกิล · 🖥️ JupyterLab Terminal → yolo1

```
ช่วยจัดทีมเอเจนต์จากบัญชีอื่นใน container นี้ (yolo1–5 และ cyolo1–5 ใช้เฉพาะบัญชีที่พร้อมใช้งาน) ทำงานต่อไปนี้ในโฟลเดอร์ปัจจุบัน
1. ให้บัญชี Claude 1 บัญชี และบัญชี Codex 1 บัญชี เขียนฟังก์ชัน Python parse_duration(text) ที่แปลง "1h30m", "45s", "2h" เป็นจำนวนวินาที คนละแบบพร้อมกัน บันทึกเป็น a_claude.py และ a_codex.py
2. ส่งทั้งสองไฟล์ให้บัญชี Codex อีกบัญชีหนึ่งเลือกแบบที่ดีกว่าและบอกจุดที่ต้องแก้
3. ส่งคำแนะนำนั้นให้บัญชี Claude แก้เป็น final.py แล้วรันทดสอบ 5 กรณี
4. สรุปตาราง บัญชี | งาน | เวลา (วินาที) | input tokens | output tokens | สำเร็จ/ล้ม ตัวเลขต้องมาจากผลจริงของแต่ละบัญชี ห้ามประมาณ
5. บอกโควตาที่เหลือของแต่ละบัญชีที่ใช้
```

เปิด JupyterLab Terminal พิมพ์ yolo1 แล้ววางพรอมต์ · ใน container ยังไม่มีสกิล

1.1 · ผลจริงก่อนมีสกิล

765.5 วินาที สำหรับงานจริงประมาณ 77 วินาที



ผลจริงรอบก่อนมีสกิล · 765.5 วินาที

JupyterLab Terminal · yolo1 (ยังไม่มีสกิล)

root@multi-account:/workspace/runs# yolo1 ← วางพรอมต์งานทีม

ทำครบทั้ง 4 ขั้นแล้ว final.py ผ่าน 5/5 กรณี และผมรันซ้ำเองก็ได้ 5/5 เหมือนกัน

ผลแต่ละขั้น

1. **เขียนโค้ดพร้อมกัน:** `yolo2` เขียน `a_claude.py` และ `cyolo1` เขียน `a_codex.py`
2. **รีวิว:** `cyolo2` เลือก **a_codex.py** เพราะใช้ regex ตัวเดียวตรวจทั้งข้อความ จึงบังคับลำดับหน่วย h→m→s และปฏิเสธหน่วยซ้ำกับข้อความแปลกปลอมท้ายสตริงได้ จุดที่ให้แก้มี 5 ข้อ:
3. เขียนรูปแบบที่รับให้ชัด
4. ระบุใน docstring ว่าอาจเกิด TypeError
5. กำหนดว่าจะรับค่าเกินช่วงอย่าง `1h90m` หรือไม่
6. จำกัดความยาวของตัวเลข
7. เพิ่มเทสต์
8. **แก้เป็น final.py:** `yolo2` แก้ครบทุกข้อ ยกเว้นข้อเพิ่มเทสต์ที่รันแบบ inline แทนการบันทึกไฟล์เทสต์ เพราะโจทย์สั่งให้เหลือไฟล์ใหม่แค่ final.py ผลทดสอบ:

กรณี	ตาดหวัง	ผล
`"1h30m"`	5400	PASS
`"45s"`	45	PASS
`"2h"`	7200	PASS
`"1h2m3s"`	3723	PASS
`"1h 30m"`	ValueError	PASS

ตารางสรุป (ตัวเลขจาก log ของแต่ละบัญชี)

บัญชี	งาน	เวลา (วินาที)	input tokens	output tokens	สำเร็จ/ล้ม
yolo2 (Claude)	เขียน a_claude.py	14.01	68,101 (ใหม่ 6 + สร้างแคช 12,102 + อ่านแคช 55,993)	1,160	สำเร็จ
cyolo1 (Codex)	เขียน a_codex.py	17.31	46,517 (มาจากแคช 43,008)	567	สำเร็จ
cyolo2 (Codex)	รีวิว ครั้งที่ 1	ไม่ได้วัด (ค้างนานกว่า 600 วินาที แล้วผมสั่งหยุด)	0 (ยังไม่ได้ส่งคำขอ)	0	ล้ม: รอ stdin ค้าง ผมจึงรันใหม่โดยปิด stdin
cyolo2 (Codex)	รีวิว ครั้งที่ 2	18.43	48,242 (มาจากแคช 43,776)	620 (reasoning 39)	สำเร็จ
yolo2 (Claude)	เขียน final.py และทดสอบ 5 กรณี	26.75	124,426 (ใหม่ 10 + สร้างแคช 13,396 + อ่านแคช 111,020)	2,815	สำเร็จ 5/5

ที่มาของเวลา:

- yolo2 ขั้น 1 ใช้ `duration_ms` ที่ Claude รายงาน เพราะตัวจับเวลาของผมพังในรอบแรก (เครื่องไม่มี `bc`)
- cyolo1 ใช้ timestamp แรกถึงสุดท้ายใน session log
- cyolo2 และ yolo2 ขั้น 3 จับเวลาจริงด้วยสคริปต์ Python ส่วน `duration_ms` ของ yolo2 ขั้น 3 ที่ Claude รายงานเองคือ 25.63

โควตาที่เหลือ (ดึงหลังงานเสร็จ)

- **yolo2 (Claude Max 5x, team):** โควตา 5 ชั่วโมงใช้ไป 0% เหลือ 100% (รีเซ็ต 8 พ.ย. 11:10 น.) และโควตารายสัปดาห์ใช้ไป 2% เหลือ 98% (รีเซ็ต 9 พ.ย. 13:00 น.)
- **cyolo1 (Codex prolite):** โควตารายสัปดาห์ใช้ไป 1% เหลือ 99% (รีเซ็ต 14 พ.ย. 12:21 น.) มีเครดิตคงเหลือ 62,500
- **cyolo2 (Codex prolite):** ใช้ไป 1% เหลือ 99% (รีเซ็ต 14 พ.ย. 12:21 น.) เครดิตคงเหลือ 62,500 เท่ากับ cyolo1 ทั้งสองบัญชีอาจใช้กลุ่มโควตาเดียวกัน

เวลาทั้งหมดเป็นเวลาไทย (+07)

ไฟล์อยู่ใน `/workspace/runs/before`:

- a_claude.py
- a_codex.py
- final.py
- logs/ (log ดิบของทุกขั้น และคำรีวิวใน review.txt)

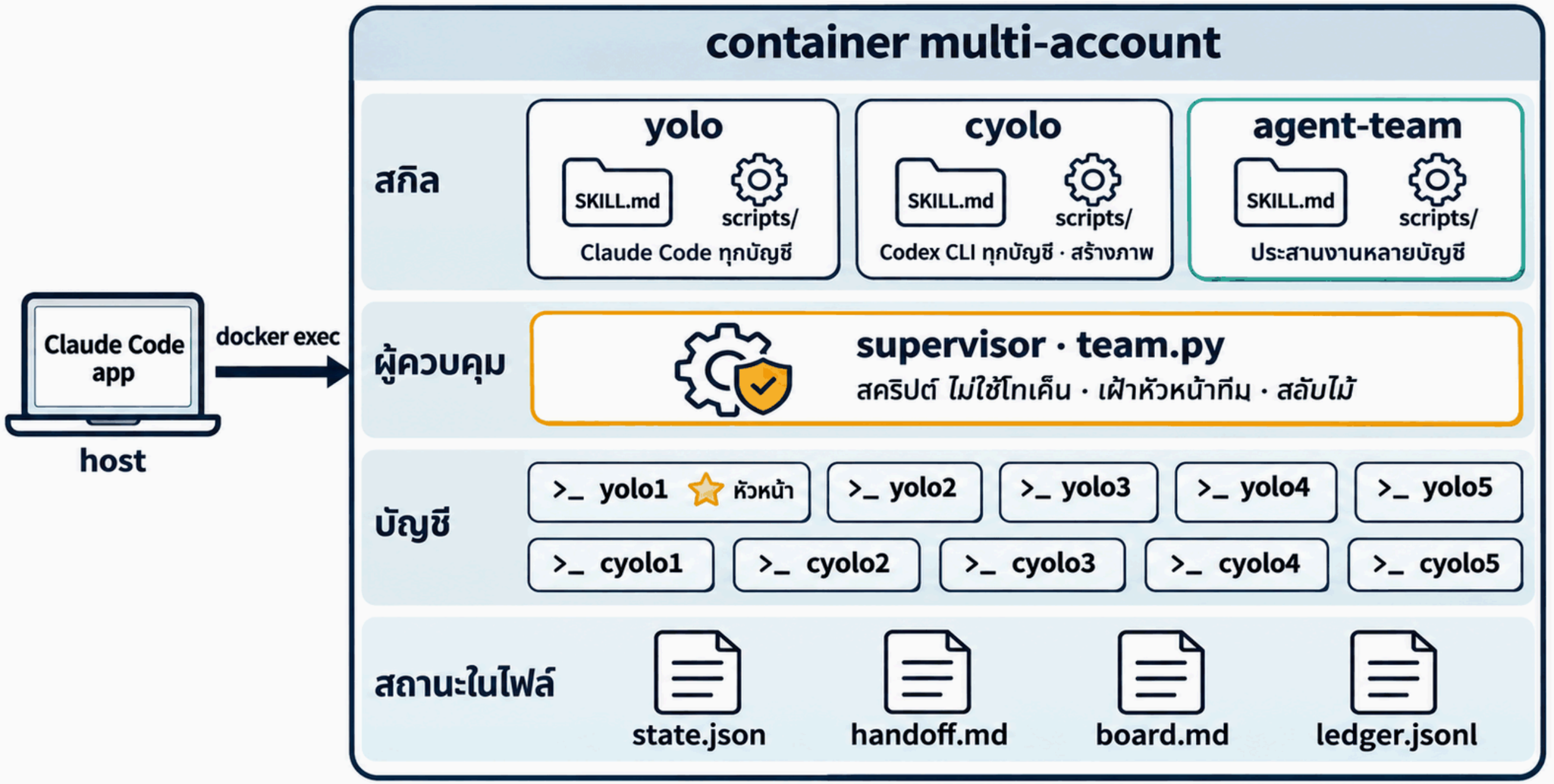
สิ่งที่สังเกตได้	รายละเอียด
สำรวจระบบเอง	4 คำสั่งแรกไล่เปิดโฟลเดอร์ อ่าน --help
อ่านไฟล์ credential เอง	เปิด .credentials.json ดูวันหมดอายุ
ค้างรอ input	codex exec ไม่ปิด stdin ค้างนานกว่า 600 วินาที
เขียนเครื่องมือเอง	ตัวจับเวลาพังรอบแรก



งานซ้ำให้สคริปต์ · งานคิดให้ AI
เรียกบัญชีให้ถูก flag ตรวจความพร้อม จับเวลา รวมสถิติ = สคริปต์ในสกิล

เปิดทีละชั้น
AI เห็นแค่ชื่อและคำอธิบายตลอดเวลา เปิด SKILL.md เมื่อจะใช้ และสคริปต์เมื่อถึงขั้นนั้น

สถาปัตยกรรมของสกิล



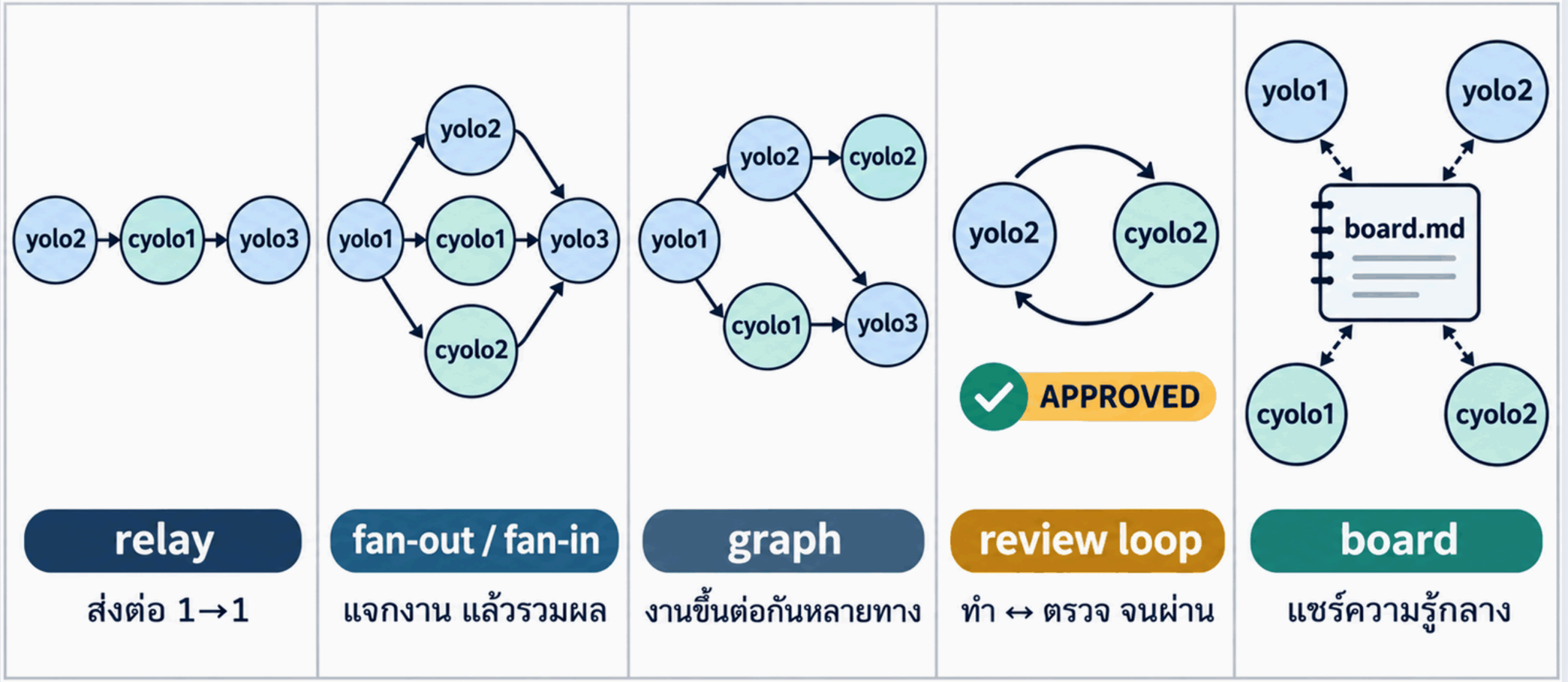
3 · ออกแบบสกิล 3 ตัว

แต่ละสกิลรู้เรื่องอะไร

สกิล	รู้เรื่องอะไร
`yolo`	Claude Code ทุกบัญชี · รุ่น/effort ตามความยาก · ตรวจความพร้อมด้วยการเรียกจริง · โควตา
`cyolo`	Codex ทุกบัญชี · รุ่นจาก models_cache · **สร้างภาพ** · โควตาจาก rollout
`agent-team`	ทำงานร่วมกันทั้งใน CLI เดียวกันและข้าม CLI · บริหารโทเค็น · สลับหัวหน้า · สถิติทั้งทีม

```
skills/
├── install.sh            ติดตั้งลง shared ของทั้งสอง CLI
├── README.md · TEST-REPORT.md
├── yolo/         SKILL.md · scripts/ · references/
├── cyolo/        SKILL.md · scripts/ · references/
└── agent-team/   SKILL.md · scripts/team.py · references/
```

รูปแบบการทำงานร่วมกันของหลายบัญชี



3.1 · รูปแบบการทำงานร่วมกัน



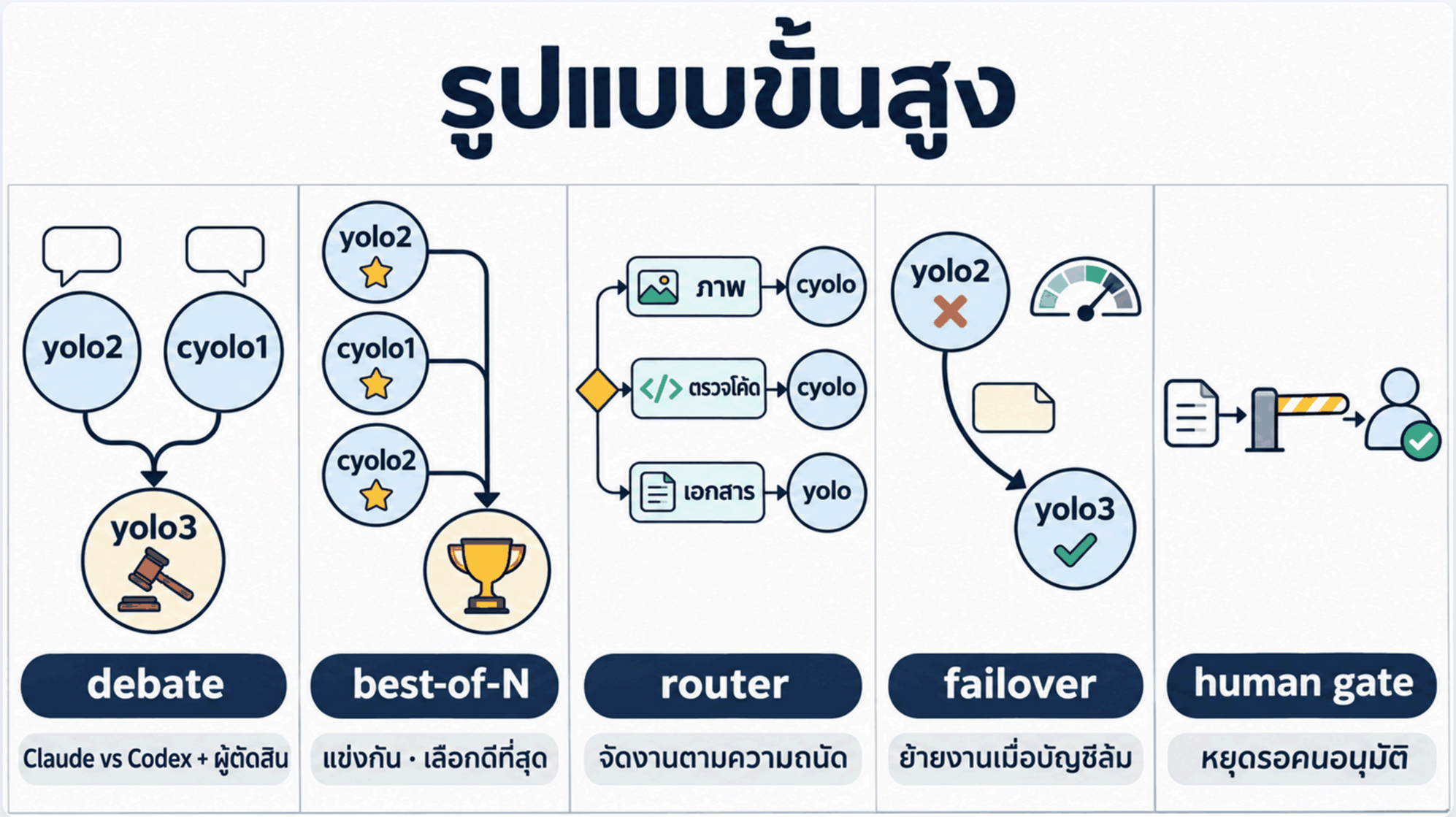
3.1 · รูปแบบการทำงานร่วมกัน

ใช้เมื่อไร และส่งข้อมูลอย่างไร

รูปแบบ	ใช้เมื่อ	การส่งข้อมูล
relay	งานเป็นลำดับ	`{{node}}` ค่าสั้นแทรกลง prompt
fan-out / fan-in	งานย่อยไม่ขึ้นต่อกัน	งานรวมผลอ่าน `{{file:node}}`
graph	งานขึ้นต่อกันหลายทาง	`depends_on` ตัวจัดคิวรันงานที่พร้อมให้เอง
review loop	ต้องการคุณภาพ	วนจน `APPROVED` หรือครบ max_rounds
board	แชร์ข้อค้นพบให้ agent ที่มาทีหลัง	อ่าน board.md ก่อนเริ่ม · เพิ่มต่อท้าย

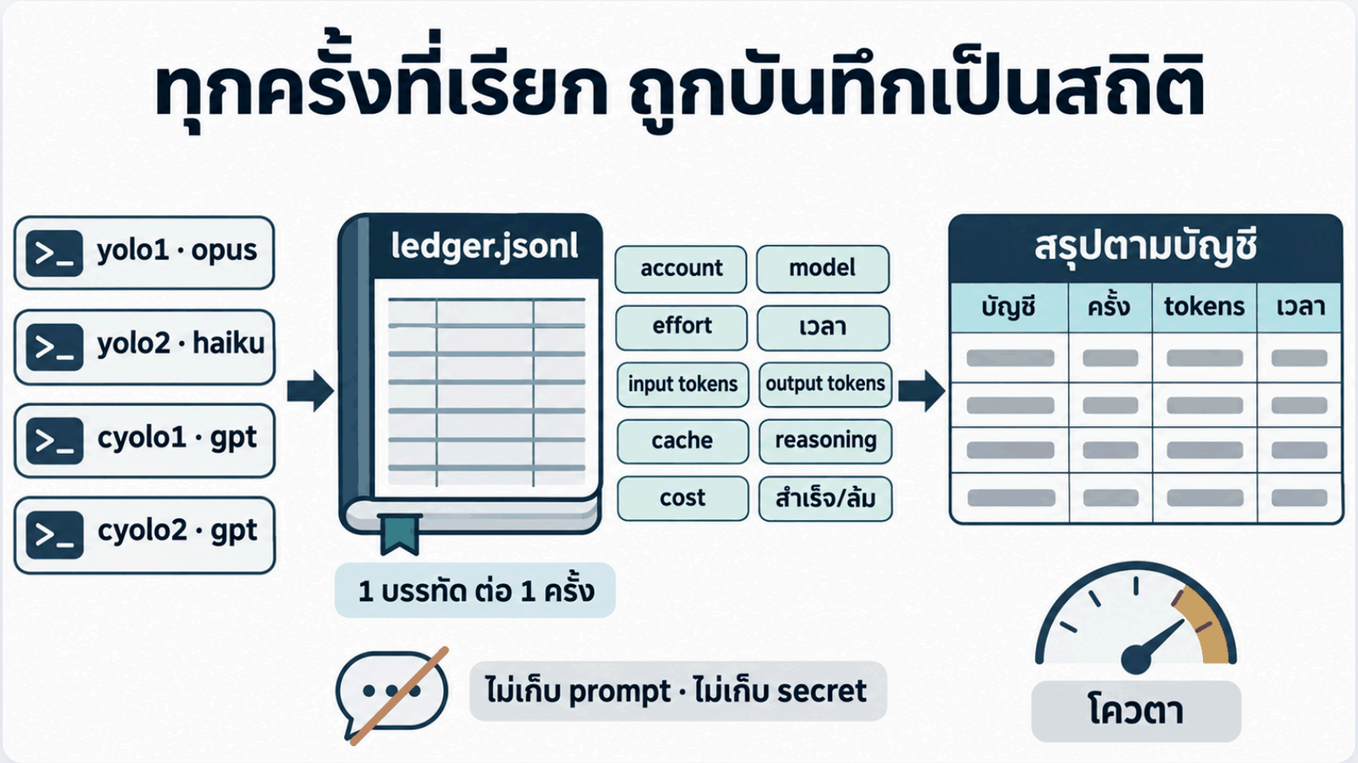
> ข้อมูลยาวส่งเป็น **path ของไฟล์** ไม่ยัดลง prompt

3.2 · รูปแบบขั้นสูง

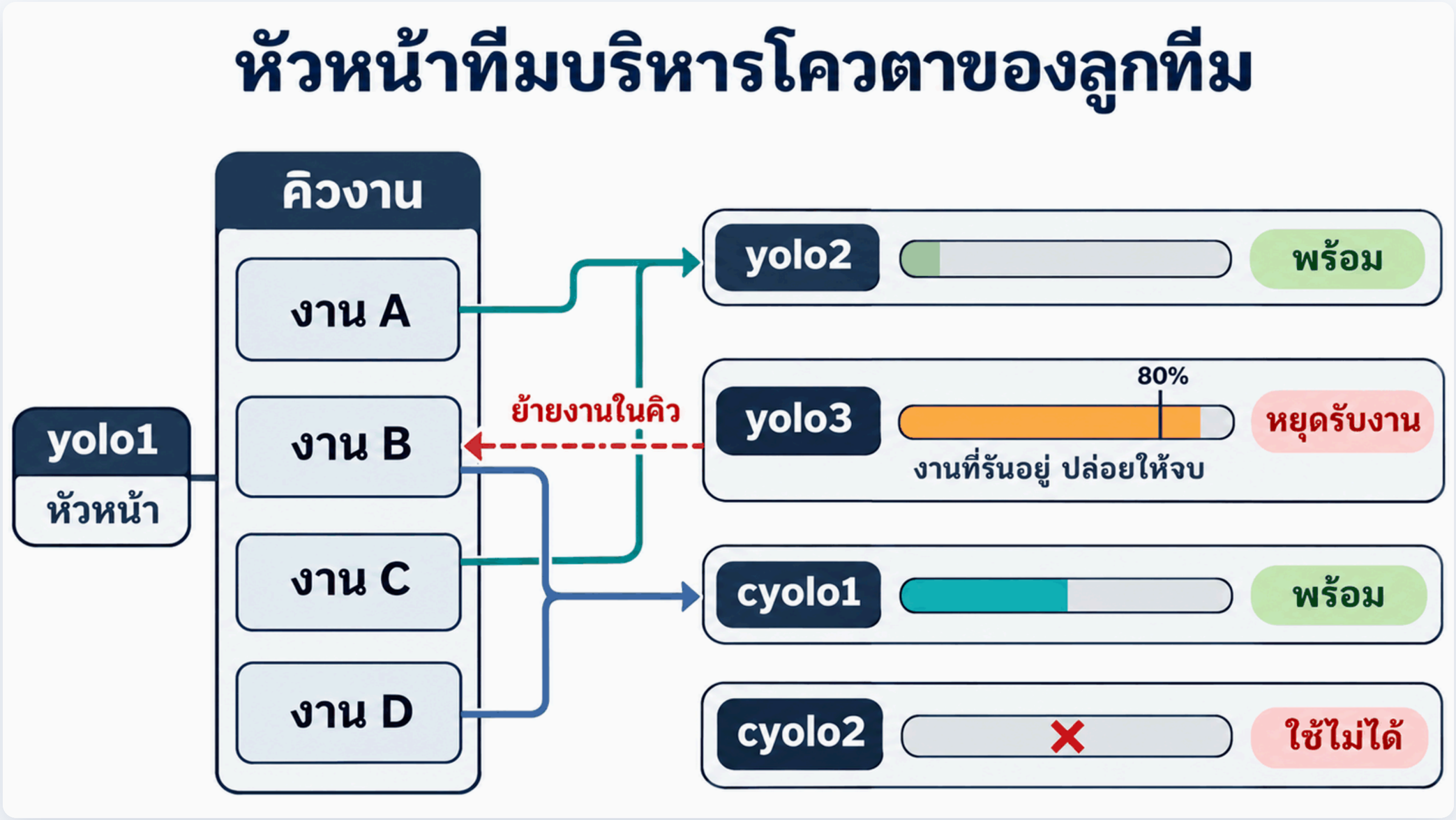


ต่อยอดได้: ทีมซ้อนทีม · โหวตเสียงข้างมาก · map-reduce · hedging · planner–executor–critic

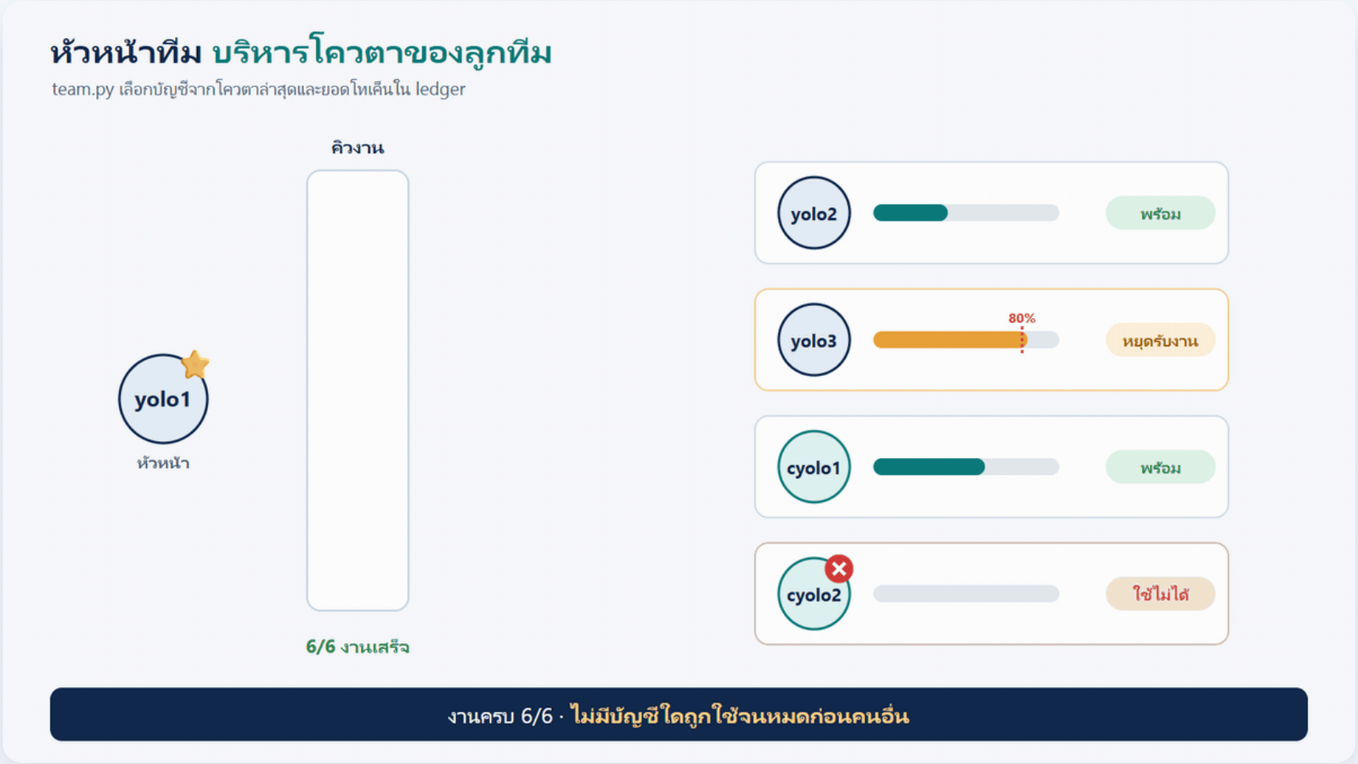
3.3 · สถิติทุกบัญชี (ledger)



- ทุกการเรียก = 1 บรรทัดใน `ledger.jsonl`
- `team.py stats` สรุปตามบัญชี / รุ่น / งาน
- `team.py quota` บอก**เวลาของข้อมูล**เสมอ
- **ไม่เก็บ prompt และ secret**


หัวหน้าทีมบริหารโควตาของลูกทีม
คิวงาน
งาน A
งาน B
งาน C
งาน D
yolo1
หัวหน้า
ย้ายงานในคิว
yolo2
พร้อม
yolo3
80%
หยุดรับงาน
งานที่รันอยู่ ปล่อยให้จบ
cyolo1
พร้อม
cyolo2
ใช้ไม่ได้

3.4 · หัวหน้าทีมบริหารโทเค็นของลูกทีม

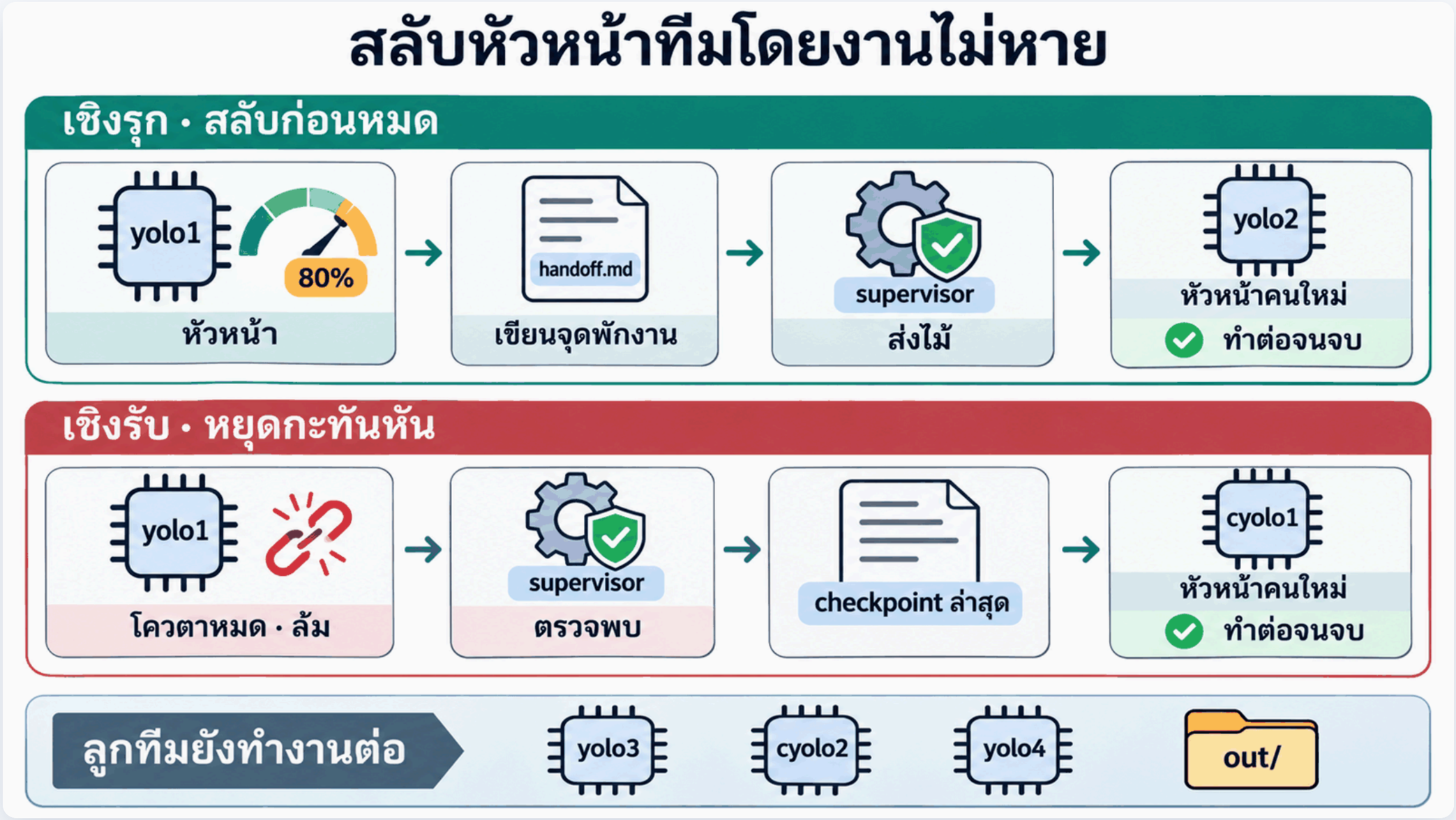


สถานการณ์	ตัวจัดคิวทำอะไร
ก่อนสั่งงาน	แจกให้บัญชีที่เหลือมากก่อน งานง่ายใช้รุ่นเล็ก
ใกล้หมด ≥ 80%	**หยุดรับงานใหม่** · งานในคิวย้าย
ล้มกลางคัน	ย้ายงาน + ความคืบหน้าใน msgs/
ผู้ใช้ระบุบัญชีเอง	ไม่ย้ายเงียบ ๆ หยุดและรายงาน

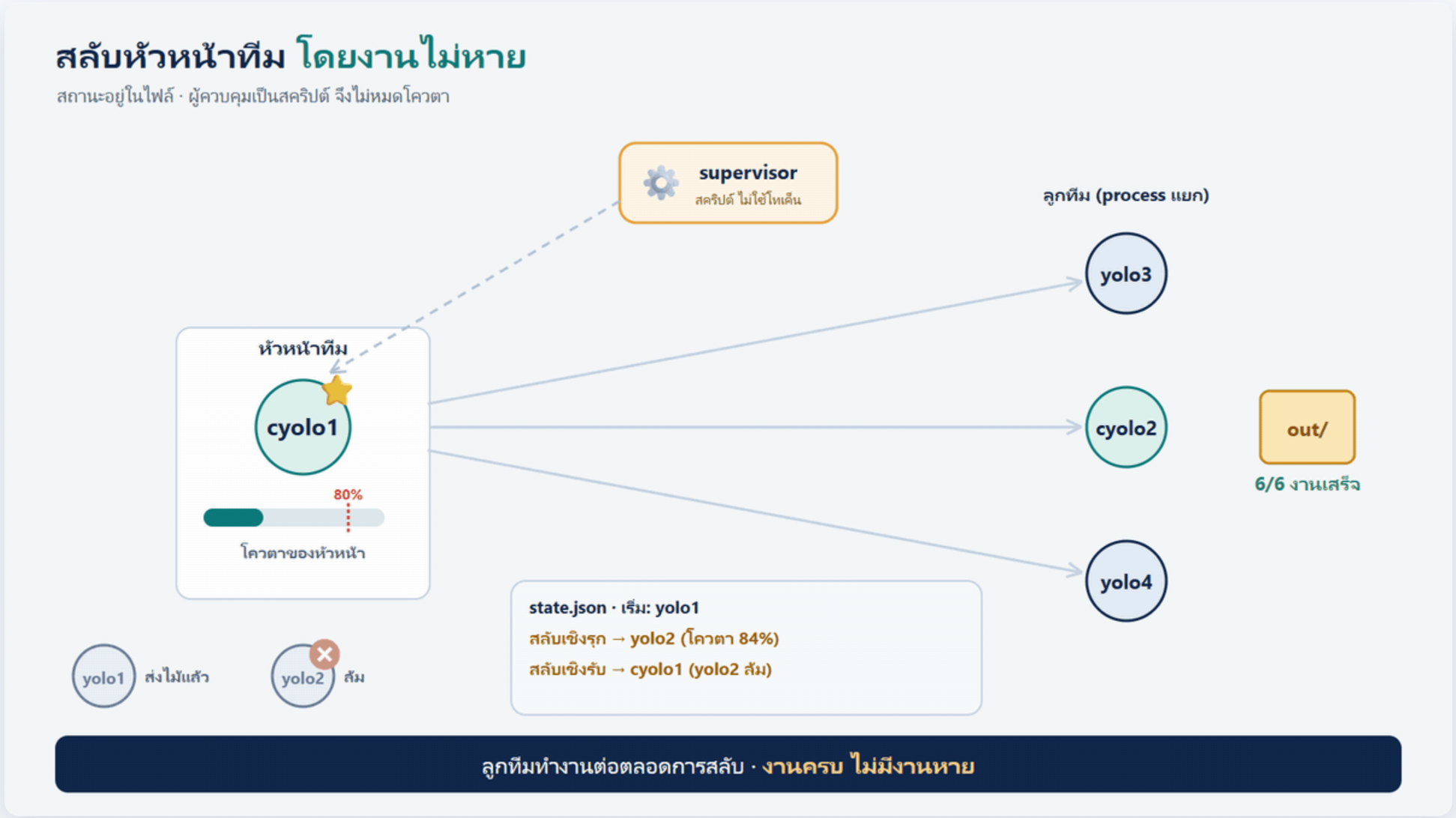


handoff.md = ใบส่งเวร

ก่อนออกเวรเขียนว่า ทำอะไรแล้ว · อะไรกำลังทำ · ต้องทำอะไรต่อ


สลับหัวหน้าทีมโดยงานไม่หาย
เชิงรุก · สลับก่อนหมด
yolo1
80%
หัวหน้า
handoff.md
เขียนจุดพักงาน
supervisor
ส่งไม้
yolo2
หัวหน้าคนใหม่
ทำต่อจนจบ
เชิงรับ · หยุดกะทันหัน
yolo1
โควตาหมด · ล้ม
supervisor
ตรวจพบ
checkpoint ล่าสุด
cyolo1
หัวหน้าคนใหม่
ทำต่อจนจบ
ลูกทีมยังทำงานต่อ
yolo3
cyolo2
yolo4
out/

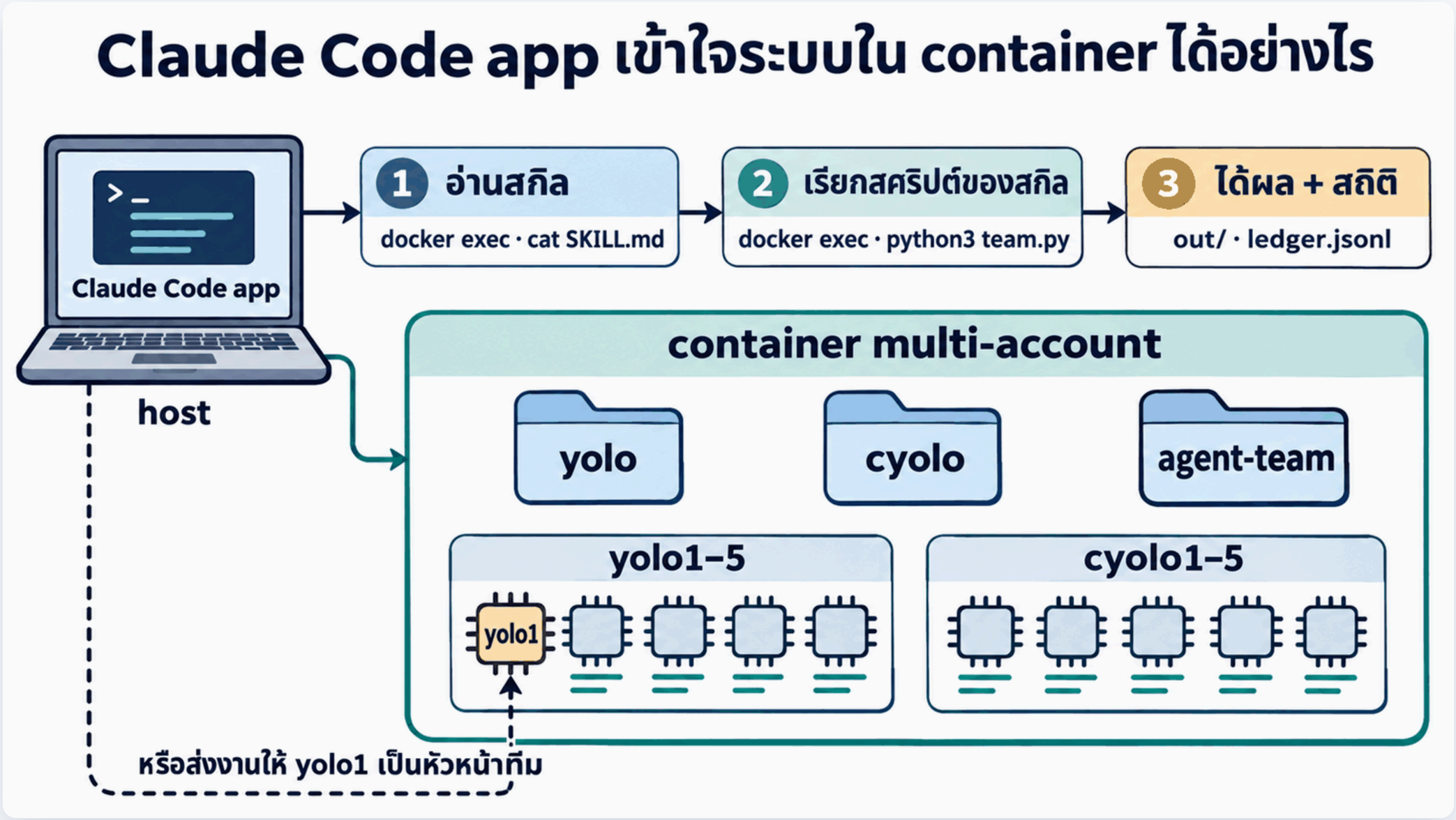
3.5 · สลับหัวหน้าทีมโดยงานไม่หาย



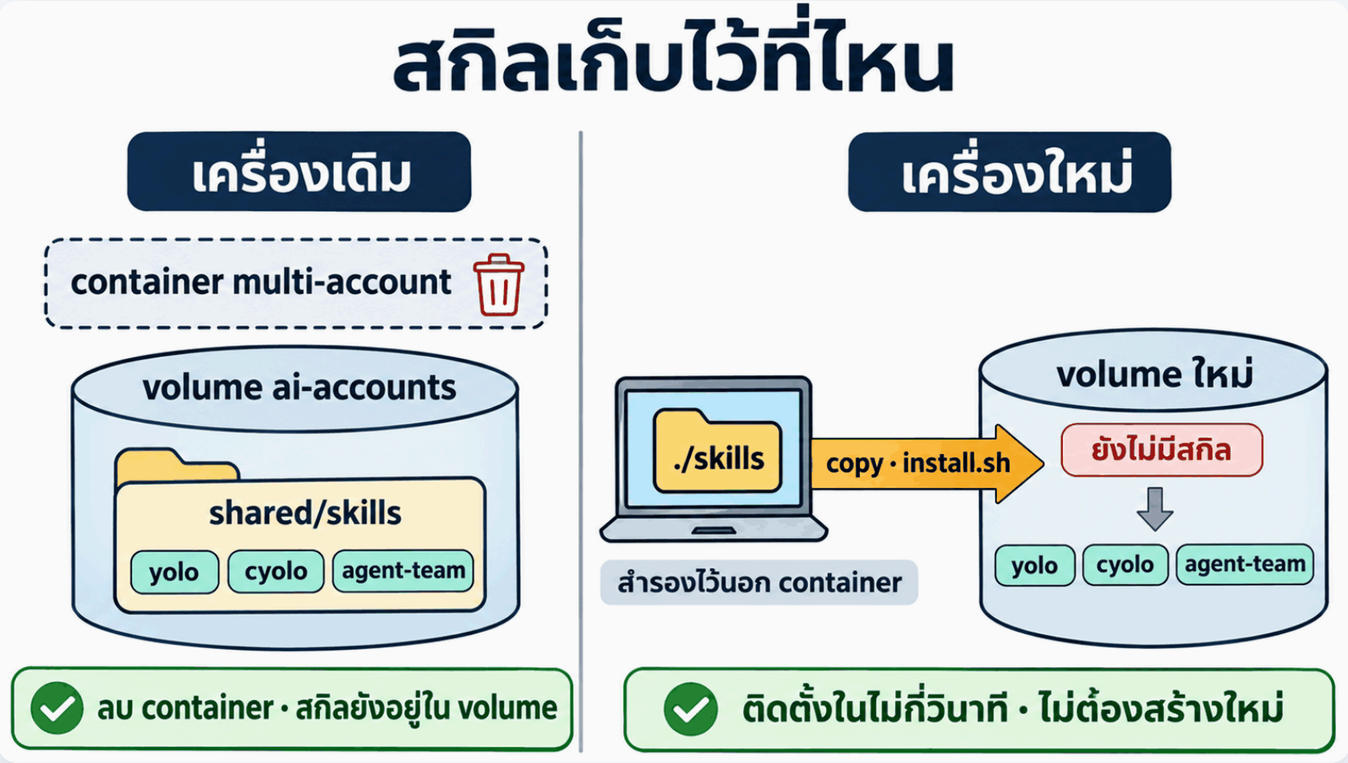
3.5 · สลับหัวหน้าทีมโดยงานไม่หาย

หลักการออกแบบ

หลักการ	รายละเอียด
สถานะอยู่ในไฟล์	state.json · handoff.md
ผู้ควบคุมเป็นสคริปต์	`team.py lead` ไม่ใช้โทเค็น · หัวหน้าทำงานเป็นรอบ เขียน checkpoint ทุกรอบ
ลูกทีมไม่ผูกกับหัวหน้า	process แยก · หัวหน้าเปลี่ยนตัว งานไม่หาย
สลับเชิงรุก	โควตา ≥ 80% หรือเกินงบ → ส่งไม้ตั้งแต่ยังเขียน checkpoint ทัน
สลับเชิงรับ	ล้ม · rate limit · เงียบไป → เริ่มคนใหม่จาก checkpoint
ผู้สืบทอด	พร้อม + โควตาเหลือมาก · ฝั่งเดียวกันก่อน (หัวหน้าเป็น Codex ได้)

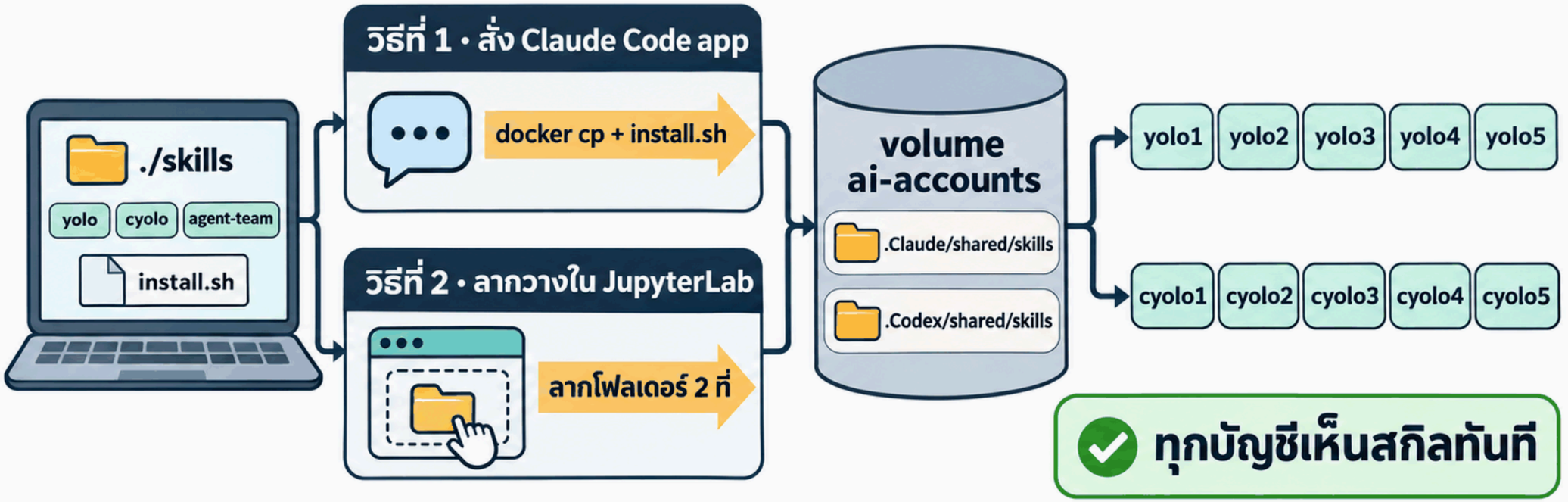

Claude Code app เข้าใจระบบใน container ได้อย่างไร
Claude Code app
host
1 อ่านสกิล
docker exec · cat SKILL.md
2 เรียกสคริปต์ของสกิล
docker exec · python3 team.py
3 ได้ผล + สถิติ
out/ · ledger.jsonl
container multi-account
yolo
cyolo
agent-team
yolo1–5
yolo1
cyolo1–5
หรือส่งงานให้ yolo1 เป็นหัวหน้าทีม

4 · สกิลเก็บไว้ที่ไหน



ที่เก็บ	ลบ container	เครื่องใหม่
โฟลเดอร์สำรอง `skills/`	ยังอยู่	**นำไปติดตั้งได้**
`shared/skills` ใน volume	ยังอยู่	ไม่ติดไป
container	หายไป	ไม่ติดไป

ติดตั้งสกิลจากโฟลเดอร์สำรอง 2 วิธี



5 · ภาพรวมขั้นตอนการทดลอง

ขั้น	ทำที่ไหน	เวลารันจริง
1 สร้างสกิลจากสเปก	Claude Code app	~33 นาที
2 ติดตั้งจากโฟลเดอร์สำรอง	Claude Code app	39 วินาที
3 งานทีมชุดเดิมอีกครั้ง	yolo1 ใน JupyterLab	134.4 วินาที
4 สลับหัวหน้า + บริหารโทเค็น	Claude Code app	~3 นาที
5 ติดตั้งบนเครื่องใหม่	Claude Code app	1.59 วินาที

```
04_MultiAccount_Skill/
├── prompt_skill_spec.md   ← สเปกของสกิล 3 ตัว (input ที่ 1)
├── reference/             ← สกิล yolo / cyolo ชุดเดิม ใช้เป็นแหล่งความรู้ (input ที่ 2)
└── _solution/skills/      ← สกิลที่ได้จากการรันจริง = โฟลเดอร์สำรองพร้อมติดตั้ง
```

ต้องมี container multi-account และล็อกอินอย่างน้อย 1 บัญชี Claude + 1 บัญชี Codex

1

ให้ Claude สร้างสกิลจากสเปก

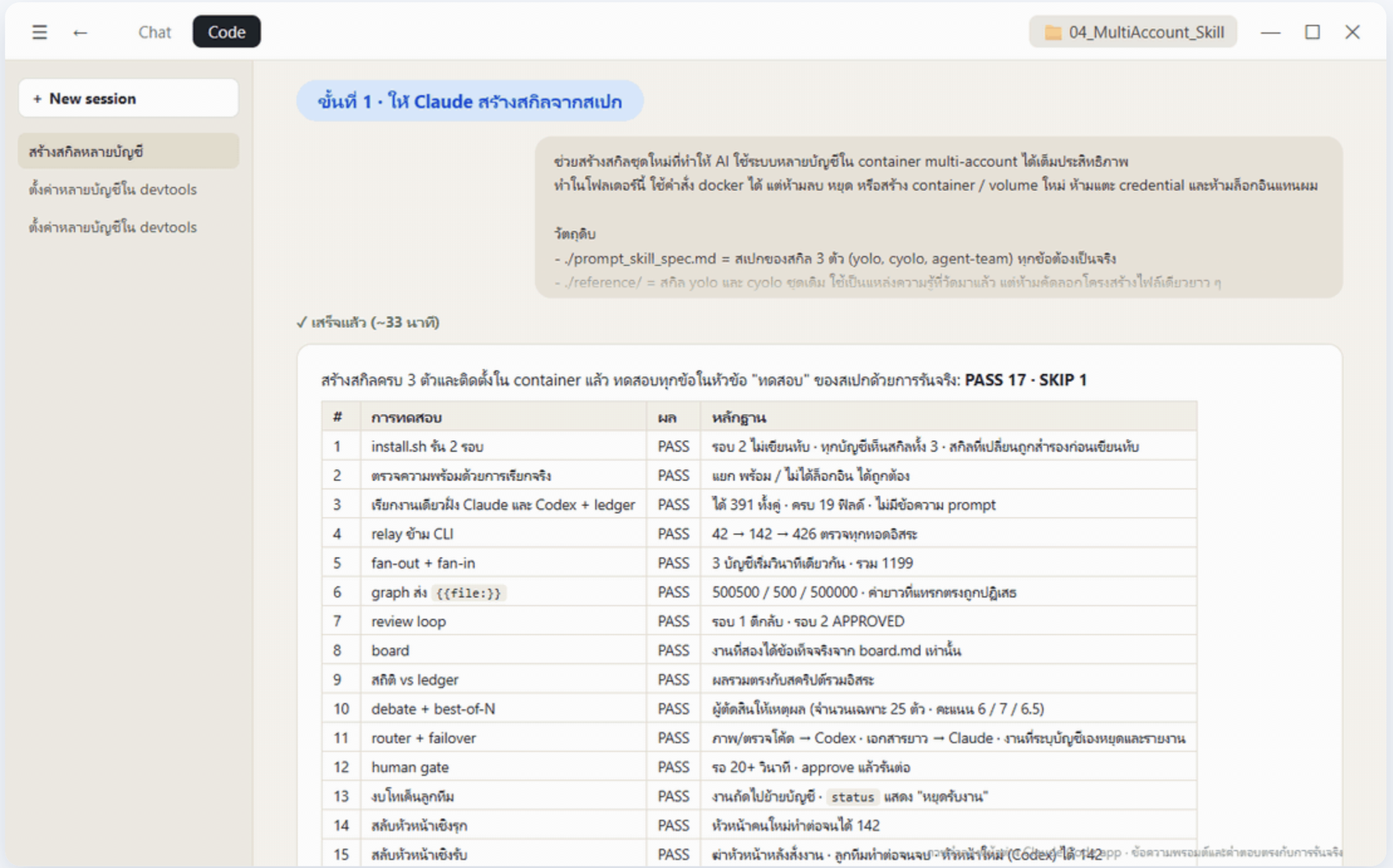
input: **prompt_skill_spec.md** + **reference/** (สกิลเดิม) → **./skills** (~33 นาที)

ขั้นที่ 1 · input

หัวข้อในสเปก	สาระ
เป้าหมาย	3 สกิล · SKILL.md + scripts/ + references/ · Python มาตรฐาน
ผู้ใช้ 2 ฝั่ง	ใน container และ Claude Code app บน host (docker exec)
ข้อเท็จจริงจากการทดลอง	flag ที่ถูก · ปิด stdin · exit code ไม่ใช่ตัวบอกผล · auth status ไม่ตรงจริง · ตำแหน่งโควตา
agent-team	run directory · พื้นฐาน 5 แบบ · ขั้นสูง 5 แบบ · บริหารโทเค็น · สลับหัวหน้า
ledger / install.sh / ทดสอบ	ฟิลด์ที่เก็บ · ติดตั้งซ้ำได้ · เกณฑ์ PASS/FAIL/SKIP

reference/ = สกิล yolo / cyolo ชุดเดิม (ไฟล์เดียวยาว ~44 KB ต่อสกิล) ใช้เป็นแหล่งความรู้

ขั้นที่ 1 · 💻 Claude Code app



ขั้นที่ 1 · พรอมต์ (คัดลอกทั้งกรอบ)

```
ช่วยสร้างสกิลชุดใหม่ที่ทำให้ AI ใช้ระบบหลายบัญชีใน container multi-account ได้เต็มประสิทธิภาพ
ทำในโฟลเดอร์นี้ ใช้คำสั่ง docker ได้ แต่ห้ามลบ หยุด หรือสร้าง container / volume ใหม่ ห้ามแตะ credential และห้ามล็อกอินแทนผม

วัตถุดิบ
- ./prompt_skill_spec.md = สเปกของสกิล 3 ตัว (yolo, cyolo, agent-team) ทุกข้อต้องเป็นจริง
- ./reference/ = สกิล yolo และ cyolo ชุดเดิม ใช้เป็นแหล่งความรู้ที่วัดมาแล้ว แต่ห้ามคัดลอกโครงสร้างไฟล์เดียวยาว ๆ
- container multi-account ที่กำลังรัน = ที่ทดสอบจริง (บัญชีที่พร้อมอาจมีไม่ครบ ให้ตรวจเอง)

สร้างไว้ใน ./skills (นอก container นี่คือต้นฉบับที่เก็บสำรอง)
ทดสอบ: คัดลอก ./skills ไปที่ /tmp/skills ใน container แล้วรัน install.sh ติดตั้งลง shared ทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปก
ใช้บัญชีอย่างประหยัด (haiku/low กับงานทดสอบ) ข้อที่ต้องใช้บัญชีที่ไม่พร้อม = SKIP
เขียน skills/TEST-REPORT.md แล้วสรุปเป็นตารางสั้น ๆ: ผลทดสอบ ไฟล์ที่สร้าง และเวลาที่ใช้ แล้วหยุด
```

ขั้นที่ 1

หลักการออกแบบพรอมต์

ให้ข้อเท็จจริงที่ตรวจแล้ว

สเปกรวมกับดักจากการรันจริง Claude ไม่ต้องลองผิดลองถูกเอง

ห้ามคัดลอกโครงสร้างเดิม

สกิลใหม่ต้องแยกชั้นตามหลักการเปิดทีละชั้น

ต้นฉบับอยู่นอก container

./skills คือโฟลเดอร์สำรองที่นำไปติดตั้งเครื่องใหม่ได้

ทดสอบแบบประหยัด

งานทดสอบใช้รุ่นเล็ก และจำลองโควตาหมดแทนการใช้จริง

ขั้นที่ 1 · ผลจริง 8 พ.ย. 2026

PASS 17 · SKIP 1

✻ **Claude** · ขั้นที่ 1 · ให้ Claude สร้างสกิลจากสเปก · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

สร้างสกิลครบ 3 ตัวและติดตั้งใน container แล้ว ทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปกด้วยการรันจริง: **PASS 17 · SKIP 1**

#	การทดสอบ	ผล	หลักฐาน
1	install.sh รัน 2 รอบ	PASS	รอบ 2 ไม่เขียนทับ · ทุกบัญชีเห็นสกิลทั้ง 3 · สกิลที่เปลี่ยนถูกสำรองก่อนเขียนทับ
2	ตรวจความพร้อมด้วยการเรียกจริง	PASS	แยก พร้อม / ไม่ได้ล็อกอิน ได้ถูกต้อง
3	เรียกงานเดียวฝั่ง Claude และ Codex + ledger	PASS	ได้ 391 ทั้งคู่ · ครบ 19 ฟิลด์ · ไม่มีข้อความ prompt
4	relay ข้าม CLI	PASS	42 → 142 → 426 ตรวจทุกทอดอิสระ
5	fan-out + fan-in	PASS	3 บัญชีเริ่มวินาทีเดียวกัน · รวม 1199
6	graph ส่ง `{{file:}}`	PASS	500500 / 500 / 500000 · ค่ายาวที่แทรกตรงถูกปฏิเสธ
7	review loop	PASS	รอบ 1 ตีกลับ · รอบ 2 APPROVED
8	board	PASS	งานที่สองได้ข้อเท็จจริงจาก board.md เท่านั้น
9	สถิติ vs ledger	PASS	ผลรวมตรงกับสคริปต์รวมอิสระ
10	debate + best-of-N	PASS	ผู้ตัดสินให้เหตุผล (จำนวนเฉพาะ 25 ตัว · คะแนน 6 / 7 / 6.5)
11	router + failover	PASS	ภาพ/ตรวจโค้ด → Codex · เอกสารยาว → Claude · งานที่ระบุบัญชีเองหยุดและรายงาน
12	human gate	PASS	รอ 20+ วินาที · approve แล้วรันต่อ
13	งบโทเคินลูกทีม	PASS	งานถัดไปย้ายบัญชี · `status` แสดง "หยุดรับงาน"
14	สลับหัวหน้าเชิงรุก	PASS	หัวหน้าคนใหม่ทำต่อจนได้ 142
15	สลับหัวหน้าเชิงรับ	PASS	ฆ่าหัวหน้าหลังสั่งงาน · ลูกทีมทำต่อจนจบ · หัวหน้าใหม่ (Codex) ได้ 142
16	เรียกจาก host ผ่าน `docker exec`	PASS	yolo.py, team.py (graph ทาง stdin), cyolo.py image
17	สร้างภาพผ่าน cyolo	PASS	PNG จริง · เป็นแมวสีส้มตามสั่ง
18	ข้อที่ต้องใช้บัญชีที่ไม่พร้อม	SKIP	ไม่มีบัญชีที่อยู่ในสถานะ "ล็อกอินแล้วแต่หมดอายุ" ให้ทดสอบ

ข้อสังเกตจากการทดสอบ

- haiku/low เคยตอบ 12+30 ผิดหนึ่งครั้ง จึงต้องตรวจผลอิสระตามสเปก
- Codex 0.167.0 ที่ไม่ได้ล็อกอินออกด้วย exit 1 (สเปกบอก 0) สคริปต์ไม่พึ่ง exit code จึงไม่กระทบ
- Git Bash บน Windows แปลง path `/root/...` ที่ส่งให้ `docker exec` เพิ่มหมายเหตุใน SKILL.md ทุกตัวแล้ว

ไฟล์ที่สร้าง (`./skills`): `install.sh` · `README.md` · `TEST-REPORT.md` · `yolo/` · `cyolo/` · `agent-team/`

ใช้งานรวม 103 ครั้ง (10 ครั้งที่ล้มเป็นการจำลองโดยตั้งใจ) · เวลา 06:38–07:10 ประมาณ 32 นาที

คำตอบจริงเป็นภาษาอังกฤษ แปลและจัดหน้าใหม่ · SKIP = ไม่มีบัญชีในสถานะ "ล็อกอินแล้วแต่หมดอายุ" ให้ทดสอบ

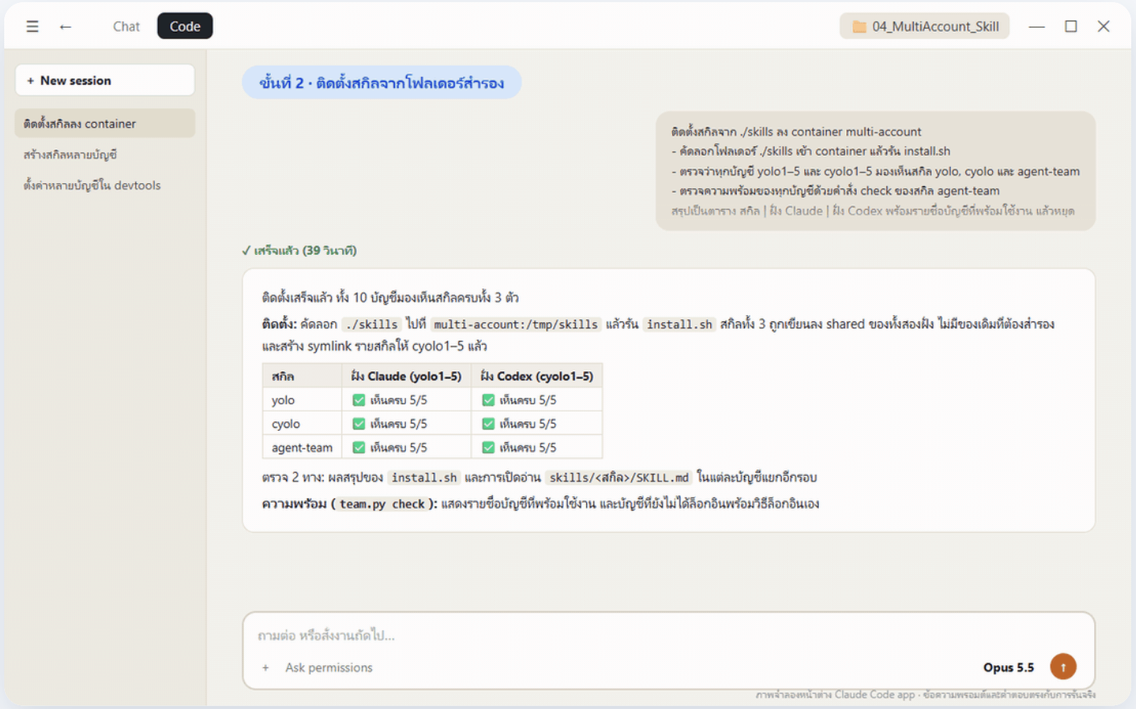
2

ติดตั้งสกิลจากโฟลเดอร์สำรอง

docker cp + **install.sh** หรือลากวางใน JupyterLab (39 วินาที)

ขั้นที่ 2 · วิธีที่ 1 · 💻 Claude Code app

```
ติดตั้งสกิลจาก ./skills ลง container multi-account
- คัดลอกโฟลเดอร์ ./skills เข้า container แล้วรัน install.sh
- ตรวจว่าทุกบัญชี yolo1-5 และ cyolo1-5 มองเห็นสกิล yolo, cyolo และ agent-team
- ตรวจความพร้อมของทุกบัญชีด้วยคำสั่ง check ของสกิล agent-team
สรุปเป็นตาราง สกิล | ฝั่ง Claude | ฝั่ง Codex พร้อมรายชื่อบัญชีที่พร้อมใช้งาน แล้วหยุด
```



ขั้นที่ 2 · ผลจริง

✻ **Claude** · ขั้นที่ 2 · ติดตั้งสกิลจากโฟลเดอร์สำรอง · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ติดตั้งเสร็จแล้ว ทั้ง 10 บัญชีมองเห็นสกิลครบทั้ง 3 ตัว

ติดตั้ง: คัดลอก `./skills` ไปที่ `multi-account:/tmp/skills` แล้วรัน `install.sh` สกิลทั้ง 3 ถูกเขียนลง shared ของทั้งสองฝั่ง ไม่มีของเดิมที่ต้องสำรอง และสร้าง symlink รายสกิลให้ cyolo1–5 แล้ว

สกิล	ฝั่ง Claude (yolo1–5)	ฝั่ง Codex (cyolo1–5)
yolo	✅ เห็นครบ 5/5	✅ เห็นครบ 5/5
cyolo	✅ เห็นครบ 5/5	✅ เห็นครบ 5/5
agent-team	✅ เห็นครบ 5/5	✅ เห็นครบ 5/5

ตรวจ 2 ทาง: ผลสรุปของ `install.sh` และการเปิดอ่าน `skills/<สกิล>/SKILL.md` ในแต่ละบัญชีแยกอีกรอบ

ความพร้อม (`team.py check`): แสดงรายชื่อบัญชีที่พร้อมใช้งาน และบัญชีที่ยังไม่ได้ล็อกอินพร้อมวิธีล็อกอินเอง

วิธีที่ 2 · ลากวางใน JupyterLab

ลาก yolo, cyolo, agent-team **ทีละโฟลเดอร์** ไปวาง 2 ที่

`.Claude/shared/skills`	yolo1–5
`.Codex/shared/skills`	cyolo1–5

ใช้สกิลที่เตรียมไว้: เปลี่ยน ./skills เป็น ./_solution/skills

3

ทดสอบงานทีมชุดเดิมอีกครั้ง

วางพรอมต์งานทีมชุดเดียวกับหัวข้อ 1.1 ใน **yolo1** (134.4 วินาที)

ขั้นที่ 3 · ผลจริงหลังมีสกิล

ผลจริงรอบหลังมีสกิล · 134.4 วินาที

JupyterLab Terminal · yolo1 (มีสกิลแล้ว)

```
root@multi-account:/workspace/runs# yolo1 ← วางพรอมต์งานทีม
```

ทำครบทั้ง 4 ขั้นแล้ว `final.py` ผ่านการทดสอบ **5/5** ผมรันไฟล์ทดสอบซ้ำเองด้วย python3 ก็ได้ผลตรงกัน

ผลแต่ละขั้น

1. **เขียนคนละแบบพร้อมกัน**: yolo2 เขียน `a_claude.py` และ cyolo1 เขียน `a_codex.py`
2. **ตรวจเลือก**: cyolo2 เลือก `a_codex.py` เพราะใช้ regex กำหนดรูปแบบชัด และปฏิเสธสตริงว่าง หน่วยซ้ำ กับหน่วยที่เรียงผิดลำดับได้อยู่แล้ว
3. cyolo2 บอกว่าไม่มีบั๊กที่จำเป็นต้องแก้ ที่เหลือเป็นข้อเสนอแนะ: ให้เขียนในคำอธิบายว่ารับเฉพาะตัวพิมพ์เล็ก, จะใช้ `strip()` หรือไม่ก็ได้, และควรเพิ่มชุดทดสอบ
4. **แก้และทดสอบ**: yolo2 แก้แค่คำอธิบาย (docstring) ให้ระบุว่ารับเฉพาะตัวพิมพ์เล็ก แล้วบันทึกเป็น `final.py`
5. ข้อเสนอที่ไม่บังคับ (`strip()` และชุดทดสอบเพิ่ม) yolo2 ไม่ได้ทำ
6. ผลทดสอบ: `"1h30m"` →5400, `"45s"` →45, `"2h"` →7200, `"1h1m1s"` →3661, `""` →ValueError ผ่านครบ

ปัญหาที่เกิดระหว่างทาง: ขั้นที่ 2 ถูกระบบตีว่าล้มในตอนแรก สาเหตุเป็นความผิดของผมเอง เพราะ regex ที่ผมตั้งไว้ตรวจรูปแบบต้องตรงกับคำตอบทั้งข้อความ คำตอบของ cyolo2 จริง ๆ ถูกต้องและใช้ได้ ผมจึงไม่สั่งให้ตรวจใหม่ (จะเสียโทเค็นซ้ำ) แต่เริ่มรอบใหม่ที่มีแค่ขั้นที่ 3 แล้วส่งไฟล์ผลตรวจเดิมให้ yolo2

ตารางสรุป

ตัวเลขทั้งหมดมาจากบันทึกการเรียกจริง (ledger) ของแต่ละบัญชี

บัญชี	งาน	เวลา (วินาที)	input tokens	output tokens	สำเร็จ/ล้ม
yolo2 (Claude sonnet)	เขียน a_claude.py	10.21	55,122	927	สำเร็จ
cyolo1 (Codex gpt-6.1-sol)	เขียน a_codex.py	21.02	49,901	694	สำเร็จ
cyolo2 (Codex gpt-6.1-sol)	ตรวจเลือกแบบที่ดีกว่า	23.62	62,539	818	สำเร็จ (ตอนแรกถูกตีว่าล้มเพราะ regex ของผม ตามที่อธิบายด้านบน)
yolo2 (Claude sonnet)	แก้เป็น final.py + ทดสอบ 5 กรณี	12.61	112,511	1,312	สำเร็จ (5/5)

ตัวเลข input tokens รวมส่วนที่ดึงจาก cache แล้ว ได้แก่ a_claude 39,070, a_codex 40,704, ตรวจ 52,736 และ final 96,900

โควตาที่เหลือ

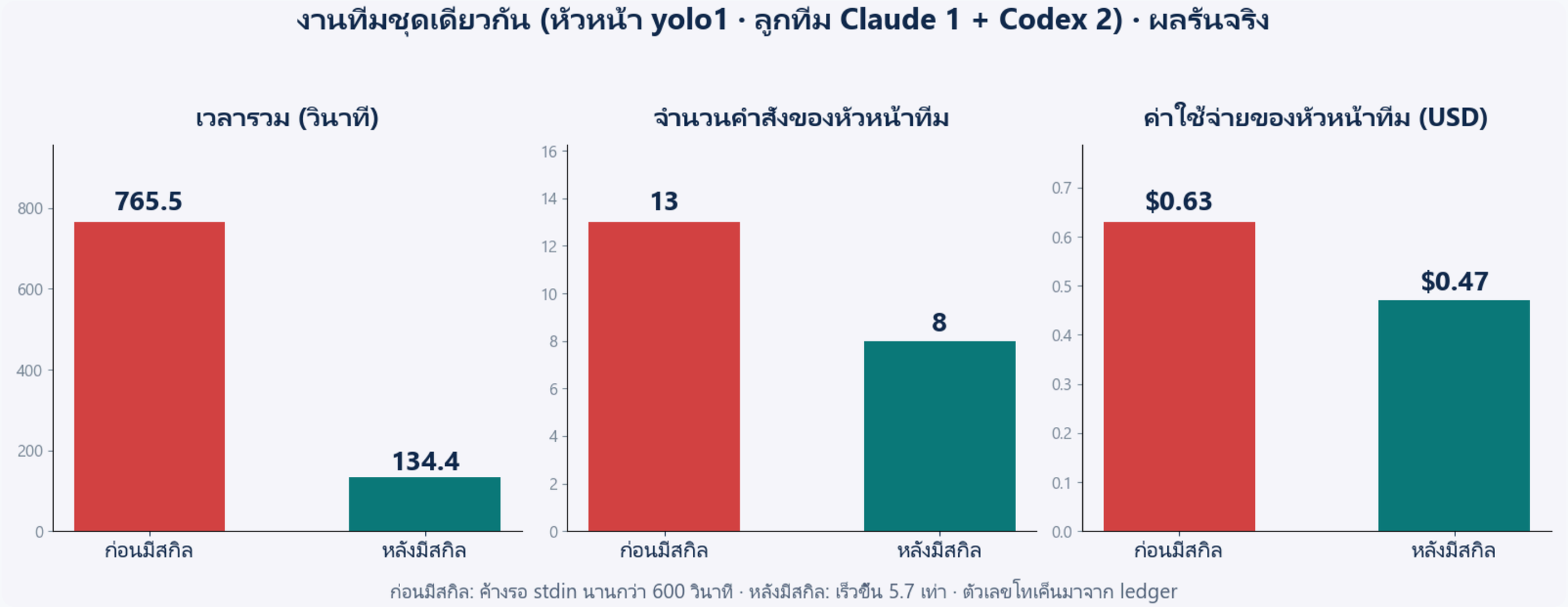
บัญชี	ใช้ไป	เหลือ	เวลาของข้อมูล
yolo1 (ผู้ประสานงาน)	สัปดาห์ (ทุกรุ่น) 22% · สัปดาห์ (Fable) 0%	~78% / 100%	07:15
yolo2	รอบนี้ (session) 1% · สัปดาห์ (ทุกรุ่น) 2% · สัปดาห์ (Fable) 1%	~98% ของสัปดาห์	07:15
cyolo1	รอบ 7 วัน 1%	~99%	06:24 (ก่อนงานนี้)
cyolo2	รอบ 7 วัน 1%	~99%	06:35 (ก่อนงานนี้)

ตัวเลขของ cyolo1 และ cyolo2 เป็นข้อมูลก่อนเริ่มงานนี้ เพราะการเรียกในงานนี้ไม่ได้บันทึกโควตาไว้ ถ้าต้องการตัวเลขล่าสุด ผมรัน `team.py quota --refresh` ได้ แต่จะต้องเรียกบัญชีละครั้งสั้น ๆ ซึ่งใช้โทเค็นเล็กน้อย

ไฟล์ `a_claude.py`, `a_codex.py` และ `final.py` อยู่ใน `/workspace/runs/after/` ส่วนบันทึกของแต่ละรอบอยู่ที่ `/workspace/.agent-team/20261108-071342-154/` และ `/workspace/.agent-team/20261108-071446-879/`

> คำสั่งแรกของ yolo1 คือ **Skill(agent-team)** · ตรวจบัญชีด้วย team.py check · สร้าง graph แล้ว team.py run · ตัวเลขทุกค่ามาจาก ledger

ขั้นที่ 3 · ก่อน vs หลังมีสกิล



ขั้นที่ 3 · ก่อน vs หลังมีสกิล

เร็วขึ้น 5.7 เท่า

	ก่อนมีสกิล	หลังมีสกิล
เวลารวม	765.5 วินาที	**134.4 วินาที**
คำสั่งแรก	สำรวจโฟลเดอร์บัญชี	Skill(agent-team)
ค้างรอ input	นานกว่า 600 วินาที	ไม่เกิด
อ่านไฟล์ credential	อ่านเอง	ไม่อ่าน
โทเค็นและโควตา	เขียนตัวจับเวลาเอง	จาก ledger + เวลาของข้อมูล

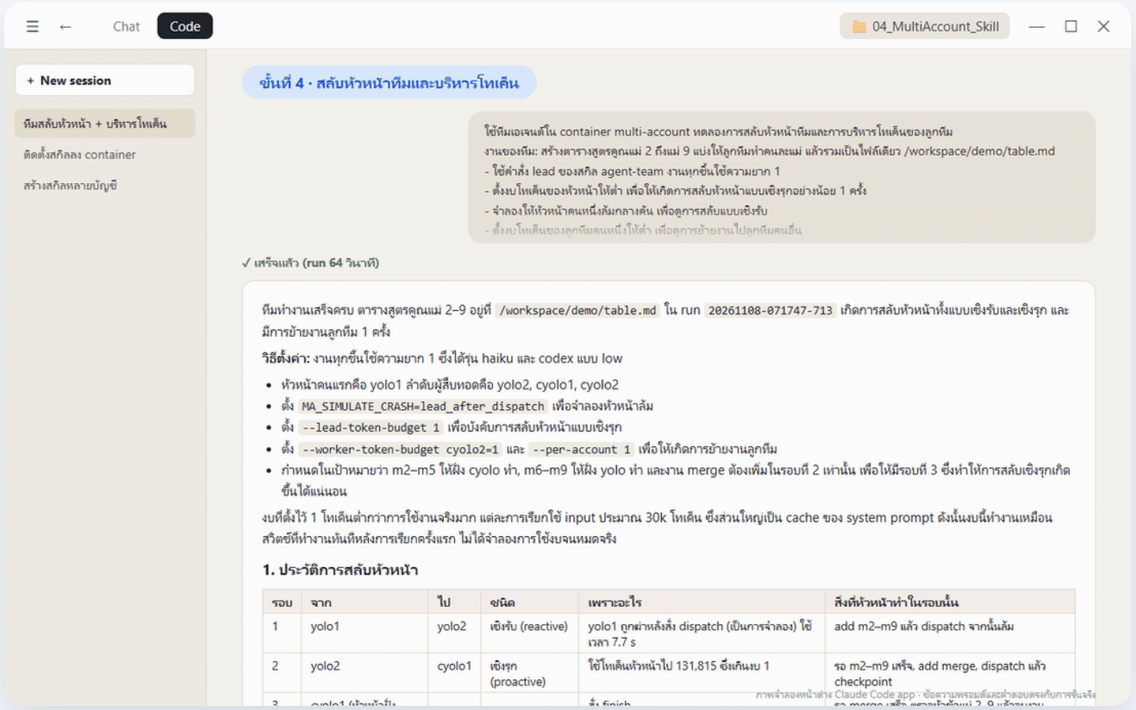
4

สลับหัวหน้าทีมและบริหารโทเค็น

team.py lead + ตัวจำลอง: หัวหน้าล้ม · หัวหน้าเกินงบ · ลูกทีมเกินงบ (~3 นาที)

ขั้นที่ 4 · 💻 Claude Code app

```
ใช้ทีมเอเจนต์ใน container multi-account ทดลองการสลับหัวหน้าทีมและการบริหารโทเค็นของลูกทีม
งานของทีม: สร้างตารางสูตรคูณแม่ 2 ถึงแม่ 9 แบ่งให้ลูกทีมทำคนละแม่ แล้วรวมเป็นไฟล์เดียว
/workspace/demo/table.md
- ใช้คำสั่ง lead ของสกิล agent-team งานทุกชิ้นใช้ความยาก 1
- ตั้งงบโทเค็นของหัวหน้าให้ต่ำ เพื่อให้เกิดการสลับหัวหน้าแบบเชิงรุกอย่างน้อย 1 ครั้ง
- จำลองให้หัวหน้าคนหนึ่งล้มกลางคัน เพื่อดูการสลับแบบเชิงรับ
- ตั้งงบโทเค็นของลูกทีมคนหนึ่งให้ต่ำ เพื่อดูการย้ายงานไปลูกทีมคนอื่น
สรุปเป็นตาราง: ประวัติการสลับหัวหน้า (จากใคร ไปใคร เพราะอะไร) การย้ายงานของลูกทีม ผลงานครบหรือไม่ และ
สถิติแยกตามบัญชี
```



ขั้นที่ 4 · ผลจริง · run 64 วินาที · 12 การเรียก

เหตุการณ์	เกิดอะไรขึ้น
สลับ**เชิงรับ**	yolo1 ล้ม (จำลอง) → yolo2 · งาน 8 ชิ้นรันต่อจนเสร็จ ไม่ถูกเพิ่มซ้ำ
สลับ**เชิงรุก**	yolo2 เกินงบหัวหน้า → cyolo1 (หัวหน้าฝั่ง Codex) จบงาน
ย้ายงานลูกทีม	cyolo2 เกินงบ → หยุดรับงาน · m5 ย้ายไป cyolo1

ตารางสูตรคูณครบ 8 แม่ 96 แถว ถูกทุกแถว · งบ 1 โทเค็นคือสวิตช์จำลอง

✻ Claude · ขั้นที่ 4 · สลับหัวหน้าทีมและบริหารโทเคิน · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ทีมทำงานเสร็จครบ ตารางสูตรคูณแม่ 2–9 อยู่ที่ `/workspace/demo/table.md` ใน run `20261108-071747-713` เกิดการสลับหัวหน้าทั้งแบบเชิงรับและเชิงรุก และมีการย้ายงานลูกทีม 1 ครั้ง

วิธีตั้งค่า: งานทุกชิ้นใช้ความยาก 1 ซึ่งได้รุ่น haiku และ codex แบบ low

- หัวหน้าคนแรกคือ yolo1 ลำดับผู้สืบทอดคือ yolo2, cyolo1, cyolo2
- ตั้ง `MA_SIMULATE_CRASH=lead_after_dispatch` เพื่อจำลองหัวหน้าล้ม
- ตั้ง `--lead-token-budget 1` เพื่อบังคับการสลับหัวหน้าแบบเชิงรุก
- ตั้ง `--worker-token-budget cyolo2=1` และ `--per-account 1` เพื่อให้เกิดการย้ายงานลูกทีม
- กำหนดในเป้าหมายว่า m2–m5 ให้ฝั่ง cyolo ทำ, m6–m9 ให้ฝั่ง yolo ทำ และงาน merge ต้องเพิ่มในรอบที่ 2 เท่านั้น เพื่อให้มีรอบที่ 3 ซึ่งทำให้การสลับเชิงรุกเกิดขึ้นได้แน่นอน

งบที่ตั้งไว้ 1 โทเคินต่ำกว่าการใช้งานจริงมาก แต่ละการเรียกใช้ input ประมาณ 30k โทเคิน ซึ่งส่วนใหญ่เป็น cache ของ system prompt ดังนั้นงบนี้ทำงานเหมือนสวิตช์ที่ทำงานทันทีหลังการเรียกครั้งแรก ไม่ได้จำลองการใช้งบจนหมดจริง

1. ประวัติการสลับหัวหน้า

รอบ	จาก	ไป	ชนิด	เพราะอะไร	สิ่งที่หัวหน้าทำในรอบนั้น
1	yolo1	yolo2	เชิงรับ (reactive)	yolo1 ถูกฆ่าหลังสั่ง dispatch (เป็นการจำลอง) ใช้เวลา 7.7 s	add m2–m9 แล้ว dispatch จากนั้นล้ม
2	yolo2	cyolo1	เชิงรุก (proactive)	ใช้โทเคินหัวหน้าไป 131,815 ซึ่งเกินงบ 1	รอ m2–m9 เสร็จ, add merge, dispatch แล้ว checkpoint
3	cyolo1 (หัวหน้าฝั่ง Codex)	—	—	สั่ง finish	รอ merge เสร็จ ตรวจหัวข้อแม่ 2–9 แล้วจบงาน

งานทั้ง 8 ชิ้นที่ yolo1 สั่งไว้ก่อนล้มยังรันต่อจนเสร็จ และไม่มีงานไหนถูกเพิ่มซ้ำ

2. การย้ายงานของลูกทีม

เวลา	เหตุการณ์	รายละเอียด
07:18:06	`worker_pause`	cyolo2 หยุดรับงานใหม่ หลังทำ m3 เสร็จ (ใช้ไป 32,691 โทเคิน เกินงบ 1)
07:18:06	`worker_switch`	งาน m5 ถูกย้ายจาก cyolo2 ไปให้ cyolo1 ทำ

3. ผลงานครบหรือไม่: ครบ

งาน	m2	m3	m4	m5	m6	m7	m8	m9	merge
ผู้ทำ	cyolo1	cyolo2	cyolo1	cyolo1 (ย้ายมา)	yolo2	yolo1	yolo2	yolo1	yolo2
สถานะ	done	done	done	done	done	done	done	done	done

ไฟล์มี 138 บรรทัด หัวข้อ `## แม่ 2` ถึง `## แม่ 9` ครบ มีแถวสูตรคูณ 96 แถว (8 แม่ × 12) ผมตรวจผลคูณทุกแถวด้วยสคริปต์แล้ว ไม่พบแถวที่ผิด

4. สถิติแยกตามบัญชี (12 การเรียก)

บัญชี	บทบาทใน run	ครั้ง	สำเร็จ / ล้ม	input (cache read)	output	reasoning	$ list	เวลารวม s
yolo1	หัวหน้ารอบ 1 + ทำ m7, m9	3	2 / 1 (ล้มคือรอบที่ถูกฆ่า)	118,254 (83,833)	1,162	419	0.0083	20.4
yolo2	หัวหน้ารอบ 2 + ทำ m6, m8, merge	4	4 / 0	334,400 (261,075)	4,013	1,287	0.0193	60.5
cyolo1	หัวหน้ารอบ 3 + ทำ m2, m4, m5	4	4 / 0	187,956 (162,560)	1,244	0	—	50.5
cyolo2	ทำ m3 แล้วหยุดรับงาน	1	1 / 0	32,442 (28,288)	249	0	—	11.2
รวม		**12**	**11 / 1**	**673,052 (535,756)**	**6,668**	**1,706**	**0.0276**	**142.5**

ค่า `$ list` คือราคาตามรายการของฝั่ง Claude ไม่ใช่ยอดที่จ่ายจริง ฝั่ง Codex ไม่มีข้อมูลค่าใช้จ่าย หลังจบงาน โควตาทุกบัญชียังเหลือมาก (ใช้ไปไม่เกิน 22%/สัปดาห์)

ไฟล์ทั้งหมดของ run (`handoff.md`, `board.md`, `state.json`, `ledger.jsonl`) อยู่ใน `/workspace/.agent-team/20261108-071747-713/`

5

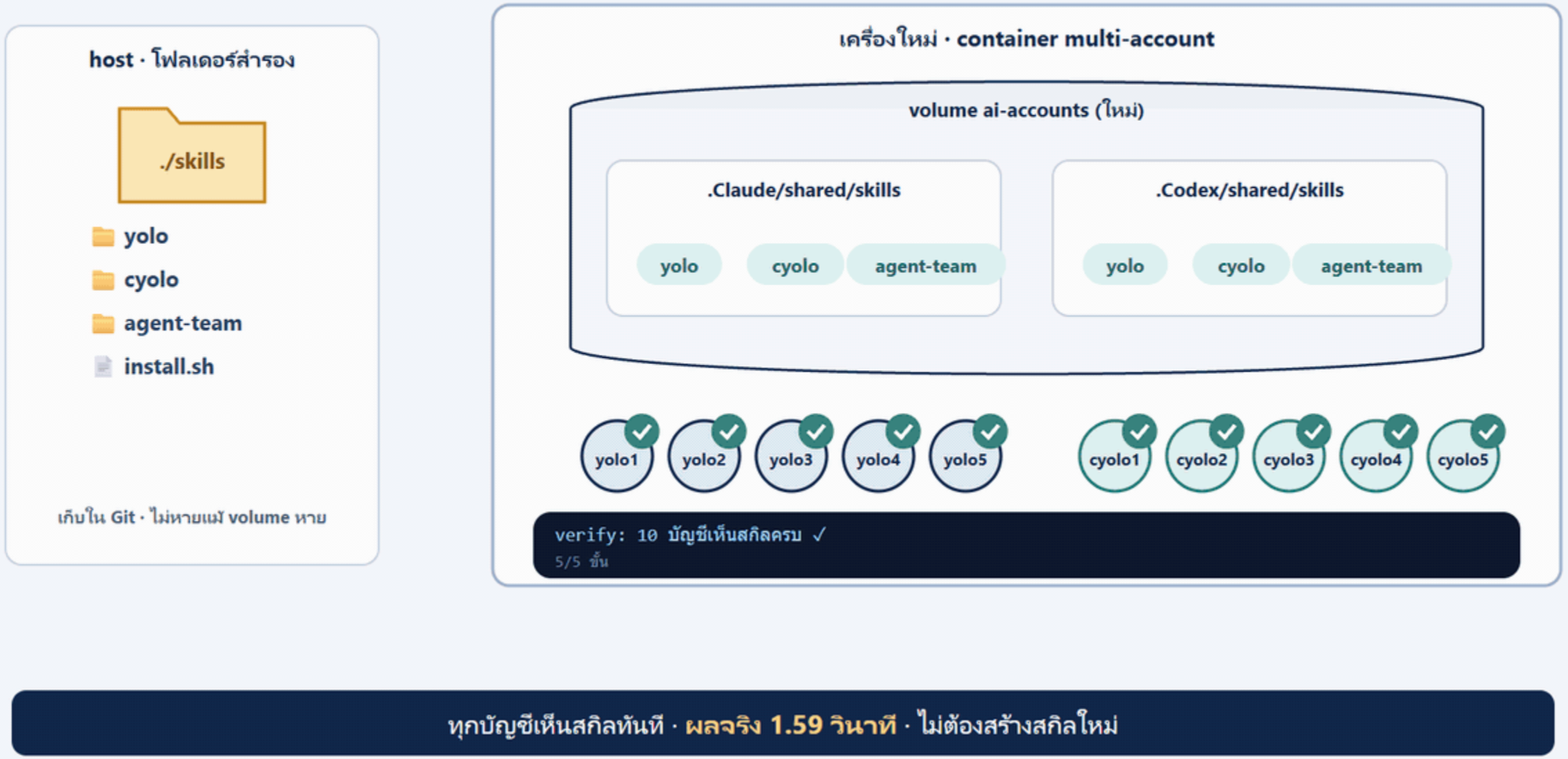
ติดตั้งบนเครื่องที่ยังไม่มี volume

volume ใหม่ไม่มีสกิล → copy โฟลเดอร์สำรอง → **1.59 วินาที**

ขั้นที่ 5 · แนวคิด

ติดตั้งสกิล บนเครื่องที่ยังไม่มี volume

ต้นฉบับสกิลอยู่นอก container จึงนำไปวางใหม่ได้ทุกเมื่อ



ทุกบัญชีเห็นสกิลทันที · ผลจริง 1.59 วินาที · ไม่ต้องสร้างสกิลใหม่

ขั้นที่ 5 · 💻 Claude Code app

จำลองเครื่องใหม่ที่ยังไม่มี volume ai-accounts แล้วติดตั้งสกิลจากโฟลเดอร์สำรอง ./skills
- สร้าง volume ai-accounts-new และ container multi-account-new จาก image multi-account:1.0 (พอร์ต 8889 · --privileged)
- ตรวจว่าใน container ใหม่ยังไม่มีสกิล yolo, cyolo, agent-team
- ติดตั้งจาก ./skills (docker cp + install.sh) และจับเวลาตั้งแต่เริ่มคัดลอกจนตรวจเสร็จ
- ตรวจว่าทุกบัญชี yolo1–5 และ cyolo1–5 มองเห็นสกิลครบ และ team.py check ทำงานได้ (บัญชีใหม่ยังไม่ได้ล็อกอินเป็นเรื่องปกติ)
- เสร็จแล้วลบ container multi-account-new และ volume ai-accounts-new ทิ้ง ห้ามแตะ container multi-account และ volume ai-accounts
สรุปเป็นตาราง ก่อนติดตั้ง | หลังติดตั้ง และเวลาที่ใช้ แล้วหยุด



ขั้นที่ 5 · ผลจริง

✻ **Claude** · ขั้นที่ 5 · ติดตั้งบนเครื่องที่ยังไม่มี volume · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ติดตั้งสกิลจาก `./skills` บนเครื่องใหม่สำเร็จ ทั้ง 10 บัญชีมองเห็นสกิลครบทั้ง 3 ตัว และ `team.py check` ทำงานได้ แต่ install.sh รอบแรกล้มกับบัญชี yolo1–5 ผมจึงแก้ `skills/install.sh` แล้วทดสอบใหม่ตั้งแต่ต้นบน volume ว่าง

รายการ	ก่อนติดตั้ง	หลังติดตั้ง
สกิลใน `.Claude/shared/skills`	มีแค่ `shared-check`	`shared-check` , `yolo` , `cyolo` , `agent-team`
สกิลใน `.Codex/shared/skills`	มีแค่ `shared-check`	`shared-check` , `yolo` , `cyolo` , `agent-team`
บัญชี yolo1–5 ที่เห็นสกิลทั้ง 3	0/5	5/5 ✅
บัญชี cyolo1–5 ที่เห็นสกิลทั้ง 3	0/5	5/5 ✅
`SKILL.md` ที่อ่านได้ (10 บัญชี × 3 สกิล)	0/30	30/30 ✅
`team.py check`	ยังไม่มีไฟล์	ทำงานได้ · บัญชีใหม่ขึ้น "ไม่ได้ล็อกอิน" เป็นเรื่องปกติ

เวลาตั้งแต่เริ่มคัดลอกจนตรวจเสร็จ: 1.59 วินาที (`docker cp` 0.39 · `install.sh` 0.55 · ตรวจทุกบัญชี 0.65)

แก้ install.sh: ใน volume ใหม่ `yoloN/skills` เป็นโฟลเดอร์จริง ไม่ใช่ symlink เหมือน volume เดิม จึงต้องสร้าง link รายสกิลให้ทั้ง yolo และ cyolo แบบเดียวกับ wrapper ของ image

เก็บกวาด: ลบ container `multi-account-new` และ volume `ai-accounts-new` แล้ว ไม่ได้แตะ container `multi-account` และ volume `ai-accounts`

1.59 วินาที · 30/30 SKILL.md อ่านได้

เจอบั๊กจริง: volume ใหม่ yoloN/skills เป็นโฟลเดอร์จริง install.sh รอบแรกจึงลิงก์ให้แค่ Codex · Claude แก้แล้วทดสอบใหม่

ทดสอบบนเครื่องใหม่จริงจำเป็น เพราะเครื่องเดิมอาจซ่อนปัญหา

สรุป

ไม่มีสกิล vs มีสกิล

	ไม่มีสกิล	มีสกิล
วิธีเรียกแต่ละบัญชี	ลองผิดลองถูก เปิด --help	สคริปต์ใส่ flag ถูกและตรวจผลให้
บัญชีไหนใช้ได้	auth status (ไม่ตรงจริง)	check ด้วยการเรียกจริง
การทำงานร่วมกัน	สั่งทีละตัว ยัดข้อมูลลง prompt	relay · fan-out · graph · review · board · ขั้นสูง
สถิติ	ต้องเดา	ledger + โควตาพร้อมเวลาของข้อมูล
โควตาหมด	งานค้างหรือหาย	สลับหัวหน้าและย้ายงาน งานไม่หาย
เครื่องใหม่	สร้างใหม่	copy โฟลเดอร์สำรอง 1.59 วินาที

ข้อควรรู้

ข้อควรรู้

ล็อกอินเองเสมอ

สกิลไม่ล็อกอินแทน และไม่อ่าน .credentials.json / .claude.json / auth.json

ข้าม permission

ใช้กับงานที่อนุญาตเท่านั้น ตรวจผลของ agent อื่นซ้ำเสมอ

ไม่ retry อัตโนมัติ

งานที่ล้มหลังใช้โทเค็นแล้วหยุดและรายงาน

Git Bash แปลง path

docker exec จาก Git Bash ใช้ `//root/...` หรือ PowerShell

แก้ที่ต้นฉบับ

แก้ใน skills/ แล้วรัน install.sh ใหม่ (สำรองของเดิมให้)

macore.py 2 สำเนา

ใน yolo และ cyolo แก้แล้วต้องให้ตรงกัน (install.sh ตรวจให้)

คำสั่งที่ใช้บ่อย

ใน container

```
T=/root/ai-accounts/.Claude/shared/skills/agent-
team/scripts/team.py
python3 $T check            # บัญชีไหนใช้ได้ (เรียกจริง)
python3 $T quota            # โควตาทุกบัญชี + เวลาของข้อมูล
python3 $T run graph.json   # รันกราฟงานจนจบ
python3 $T status           # ภาพรวมทีม
python3 $T stats            # สถิติตามบัญชี / รุ่น / run
```

ติดตั้งเอง (จากโฟลเดอร์บท)

```
docker cp ./_solution/skills multi-account:/tmp/skills
docker exec multi-account bash /tmp/skills/install.sh
```

จาก host

```
docker exec multi-account cat /root/ai-
accounts/.Claude/shared/skills/agent-team/SKILL.md
docker exec multi-account python3 /root/ai-
accounts/.Claude/shared/skills/agent-team/scripts/team.py
stats
```

เช็กลิสต์

ทำครบหรือยัง?

- [ ] มีโฟลเดอร์ `skills/` ที่มี yolo, cyolo, agent-team (ขั้นที่ 1)
- [ ] ทุกบัญชีเห็นสกิลครบ (ขั้นที่ 2)
- [ ] งานทีมชุดเดิมเร็วขึ้น และตัวเลขมาจาก ledger (ขั้นที่ 3)
- [ ] เห็นการสลับหัวหน้าทั้งเชิงรุก/เชิงรับ และการย้ายงานลูกทีม (ขั้นที่ 4)
- [ ] ติดตั้งบน volume ใหม่จากโฟลเดอร์สำรองได้ (ขั้นที่ 5)

สรุป

สอนครั้งเดียว ทุกบัญชีทำงานเป็นทีม

ความรู้อยู่ใน **สกิล** · สถานะอยู่ใน **ไฟล์** · ต้นฉบับอยู่ใน **โฟลเดอร์สำรอง**
หัวหน้าหรือลูกทีมโควตาหมด งานก็ไม่หาย

รายละเอียดทุกขั้นตอนและพรอมต์เต็มอยู่ใน README.md

